

Zadanie 4: obliczenia ewolucyjne

Termin realizacji: 4 maja 2026

Zadanie do oddania przez MS Teams. Do oddania: kod oraz krótkie sprawozdanie w PDF (można na przykład przy użyciu `quarto render notebook.ipynb --to pdf`).

Na 3.0

Do realizacji:

1. Zmodyfikuj przykład “iterative method” z notatnika `tsp_lazy_constraints.ipynb` tak, aby rozwiązywał [problem podróżującego konsumenta](#). Przykładowe problemy generuje poniższa funkcja `generate_random_tpp`. Przyjmij, że wszystkie miasta muszą być odwiedzone niezależnie od tego czy dokonany tam będzie zakup czy też nie. W rozwiązaniu wypisz w którym mieście kupione będą poszczególne przedmioty oraz narysuj trasę.
2. Zmodyfikuj analogicznie kod z notatnika `tsp-genetic.ipynb` tak aby rozwiązywał ten sam problem. Porównaj wydajność (czas obliczeń i wartość funkcji kosztu) dla dziewięciu losowych problemów, po trzy dla rozmiarów $(n=50, k=20)$, $(n=50, k=50)$ oraz $(n=100, k=100)$. Dobierz parametry algorytmu genetycznego tak, aby czas działania nie przekraczał w żadnym przypadku minuty.

Na 4.0

Do realizacji:

1. Punkty z zadania na 3.0.
2. Dodaj do problemu wagę przedmiotów, powiększającą koszt rozwiązania o długość trasy od miejsca zakupu do końca trasy pomnożoną przez wagę przedmiotu i stały współczynnik dobrany tak, aby dla przedmiotu średniej wagi jego transport między najbardziej odległymi miastami (w linii prostej) wynosił około 1% ceny. Nie dodawaj tego składnika do funkcji kosztu zaimplementowanej w JuMP, ale uwzględnij go w algorytmie genetycznym. Porównaj jakość rozwiązań uzyskanych przez oba algorytmy z uwzględnieniem wagi i bez.

3. Zaimplementuj dodatkowo [krzyżowanie PMX](#) i porównaj wydajność z tym opartym o inwersję permutacji.

Na 5.0

Do realizacji:

1. Punkty z zadania na 4.0.
2. Zmodyfikuj problem tak, aby konsument nie musiał odwiedzać miast w których niczego nie kupuje.
3. Dodaj dodatkowo do porównania zmodyfikowany dla problemu podróżującego konsumenta przykład “lazy constraints” z notatnika `tsp_lazy_constraints.ipynb`. Dane do problemu powinny być generowane tak, aby konsument w zwracanym rozwiązaniu odwiedzał przynajmniej 3 miasta i omijał przynajmniej 2.

Autorzy: Paweł Knot, Jarosław Klima, Szymon Kowalski

```
import Pkg;
Pkg.add("JuMP")
Pkg.add("GLPK")
Pkg.add("DataFrames")
# Pin Evolutionary to a known working patch release. Use 0.10.1 explicitly
Pkg.add(name="Evolutionary", version="0.10.1")
```

```
Resolving package versions...
```

```
Project No packages added to or removed from `C:\Users\szyro\.julia\environments\v1.12\l
```

```
Manifest No packages added to or removed from `C:\Users\szyro\.julia\environments\v1.12\l
```

```
Resolving package versions...
```

```
Project No packages added to or removed from `C:\Users\szyro\.julia\environments\v1.12\l
```

```
Manifest No packages added to or removed from `C:\Users\szyro\.julia\environments\v1.12\l
```

```
Resolving package versions...
```

```
Project No packages added to or removed from `C:\Users\szyro\.julia\environments\v1.12\l
```

```
Manifest No packages added to or removed from `C:\Users\szyro\.julia\environments\v1.12\l
```

```
Resolving package versions...
```

```
Project No packages added to or removed from `C:\Users\szyro\.julia\environments\v1.12\l
```

```
Manifest No packages added to or removed from `C:\Users\szyro\.julia\environments\v1.12\l
```

ZADANIE na 3.0

```
using JuMP
import GLPK
import Random
import Plots

function generate_random_tpp(n, k; random_seed=1)
    rng = Random.MersenneTwister(random_seed)
    # distance matrix
    X = 100 * rand(rng, n)
    Y = 100 * rand(rng, n)
    d = [sqrt((X[i] - X[j])^2 + (Y[i] - Y[j])^2) for i in 1:n, j in 1:n]

    cost_item_city = 50 .+ 100 .* rand(rng, k, n) # cost_item_city[num_item, num_city]

    return X, Y, d, cost_item_city
end

function generate_random_tpp(n, k, random_seed)
    rng = Random.MersenneTwister(random_seed)
    # distance matrix
    X = 100 * rand(rng, n)
    Y = 100 * rand(rng, n)
    d = [sqrt((X[i] - X[j])^2 + (Y[i] - Y[j])^2) for i in 1:n, j in 1:n]

    cost_item_city = 50 .+ 100 .* rand(rng, k, n) # cost_item_city[num_item, num_city]

    return X, Y, d, cost_item_city
end

n = 50
k = 20
X, Y, d, cost_item_city = generate_random_tpp(n, k)
```

([9.913970137863682, 70.19797138879542, 50.3261785841856, 87.58412053070398, 95.346541187448

Metoda iteracyjna

```
function build_tpp_model(d, n, cost_item_city, m)
    model = Model(GLPK.Optimizer)

    @variable(model, x[1:n, 1:n], Bin, Symmetric)
    @variable(model, z[1:m, 1:n], Bin)

    @objective(model, Min, (sum(d .* x)/2) + sum(cost_item_city .* z))

    @constraint(model, [i in 1:n], sum(x[i, :]) == 2)
    @constraint(model, [i in 1:n], x[i, i] == 0)

    @constraint(model, [k in 1:m], sum(z[k, :]) == 1)
    return model
end
```

build_tpp_model (generic function with 1 method)

```
function subtour(edges::Vector{Tuple{Int,Int}}, n)
    shortest_subtour, unvisited = collect(1:n), Set{Int}(collect(1:n))
    while !isempty(unvisited)
        this_cycle, neighbors = Int[], unvisited
        while !isempty(neighbors)
            current = pop!(neighbors)
            push!(this_cycle, current)
            if length(this_cycle) > 1
                pop!(unvisited, current)
            end
            neighbors =
                [j for (i, j) in edges if i == current && j in unvisited]
        end
        if length(this_cycle) < length(shortest_subtour)
            shortest_subtour = this_cycle
        end
    end
    return shortest_subtour
end
```

subtour (generic function with 1 method)

```
function selected_edges(x::Matrix{Float64}, n)
    return Tuple{Int,Int}[(i, j) for i in 1:n, j in 1:n if x[i, j] > 0.5]
end
```

selected_edges (generic function with 1 method)

```
subtour(x::Matrix{Float64}) = subtour(selected_edges(x, size(x, 1)), size(x, 1))
subtour(x::AbstractMatrix{VariableRef}) = subtour(JuMP.value.(x))
```

subtour (generic function with 3 methods)

```
iterative_model = build_tpp_model(d, n, cost_item_city, k)
optimize!(iterative_model)
@assert is_solved_and_feasible(iterative_model)
time_iterated = solve_time(iterative_model)
cycle = subtour(iterative_model[:x])
while 1 < length(cycle) < n
    println("Found cycle of length $(length(cycle))")
    S = [(i, j) for (i, j) in Iterators.product(cycle, cycle) if i < j]
    @constraint(
        iterative_model,
        sum(iterative_model[:x][i, j] for (i, j) in S) <= length(cycle) - 1,
    )
    optimize!(iterative_model)
    @assert is_solved_and_feasible(iterative_model)
    global time_iterated += solve_time(iterative_model)
    global cycle = subtour(iterative_model[:x])
end

objective_value(iterative_model)
```

```
Found cycle of length 3
Found cycle of length 10
Found cycle of length 3
Found cycle of length 3
```

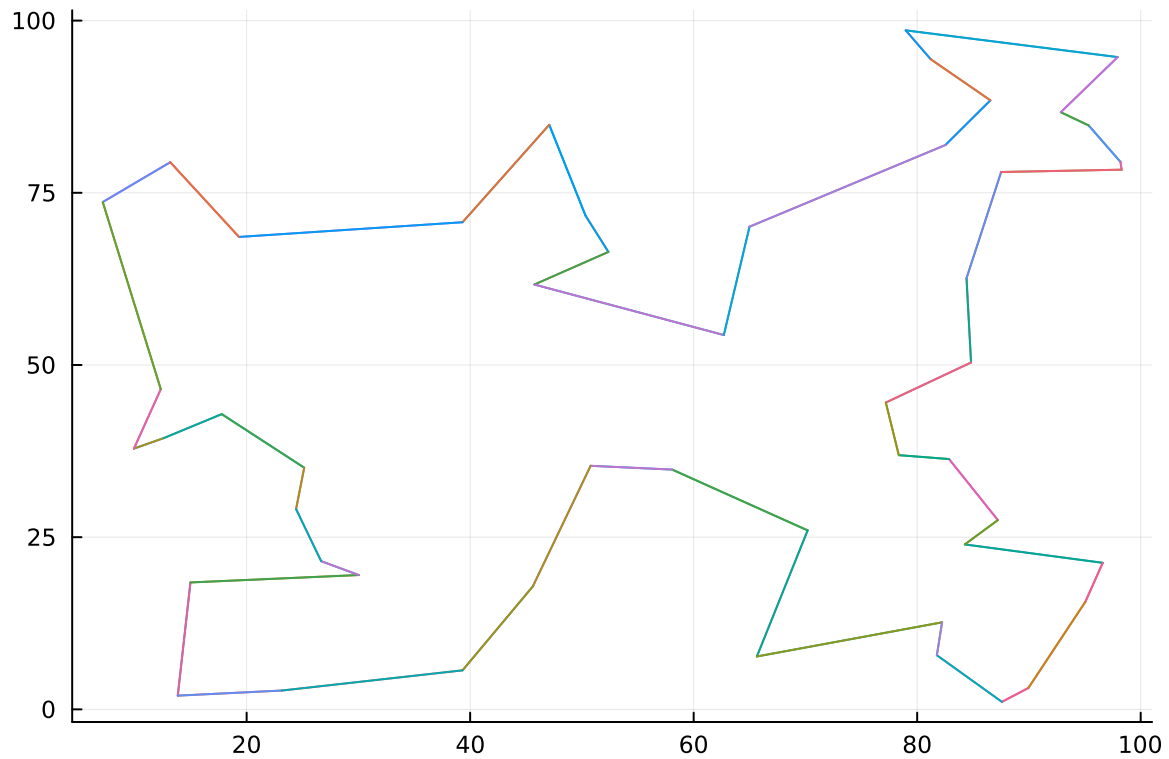
```
1569.9524382246184
```

```

function plot_tour(X, Y, x)
    plot = Plots.plot()
    for (i, j) in selected_edges(x, size(x, 1))
        Plots.plot!([X[i], X[j]], [Y[i], Y[j]]); legend = false)
    end
    return plot
end

plot_tour(X, Y, value.(iterative_model[:x]))

```



```

function plot_tour_with_shopping(model,X,Y,n,k)
    x_val = value.(model[:x])
    z_val = value.(model[:z])

    p = Plots.plot(legend=false)
    for (i,j) in selected_edges(x_val,n)
        Plots.plot!([X[i],X[j]], [Y[i],Y[j]])
    end
end

```

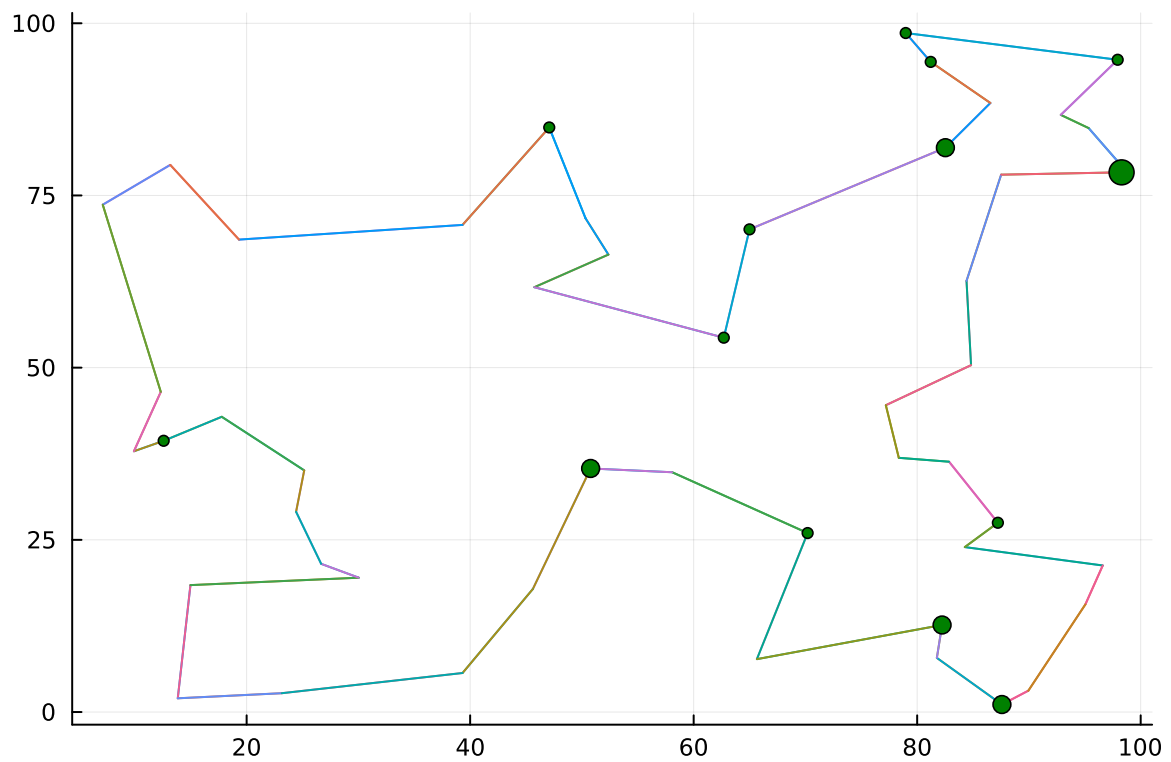
```

for i in 1:n
    items_count = sum(z_val[:,i])
    if items_count>0
        msize = 1 + 2*items_count
        Plots.scatter!([X[i]], [Y[i]],
            color=:green,
            markersize=msize,
            label="")
    end
end
return p
end

```

plot_tour_with_shopping (generic function with 1 method)

```
plot_tour_with_shopping(iterative_model,X,Y,n,k)
```



```

using DataFrames
using Evolutionary
function get_shopping_df(model,n,k)
    z_val = value.(model[:z])
    df = DataFrame(Produkt = Int[], Miasto = Int[], Cena = Float64[])

    for p in 1:k, i in 1:n
        if z_val[p,i] > 0.5
            push!(df,(p,i,cost_item_city[p,i]))
        end
    end
    show(df,allrows=true, show_row_number=false,eltypes=false,summary=false)
end

```

get_shopping_df (generic function with 1 method)

```
get_shopping_df(iterative_model,n,k)
```

Produkt	Miasto	Cena
1	46	56.9338
2	33	50.9831
3	41	51.6528
4	25	50.391
5	13	50.3309
6	16	50.7352
7	16	50.496
8	16	51.1225
9	6	51.108
10	19	50.1019
11	2	50.8029
12	4	50.7244
13	4	50.9559
14	40	50.2427
15	25	52.4366
16	13	50.8869
17	22	50.1311
18	7	52.8782
19	14	51.0647
20	6	50.1949

Metoda genetyczna

```
using Evolutionary
using Random

function perm_to_inv(perm::Vector)
    N = length(perm)
    inv = zeros{Int, N}
    for i in 1:N
        m = 1
        while perm[m] != i
            if perm[m] > i
                inv[i] += 1
            end
            m += 1
        end
    end
    return inv
end

function inv_to_perm(inv::Vector)
    N = length(inv)
    pos = zeros{Int, N}
    for i in N:-1:1
        for m in (i+1):N
            if pos[m] >= inv[i] + 1
                pos[m] += 1
            end
        end
        pos[i] = inv[i] + 1
    end
    perm = zeros{Int, N}
    for i in 1:N
        perm[pos[i]] = i
    end
    return perm
end

RP = randperm(20)
iRP = perm_to_inv(RP)
println(RP == inv_to_perm(iRP))
```

true

```
function has_subtour(perm::Vector)
    c = 1
    for i in 1:(length(perm)-1)
        c = perm[c]
        if c == 1
            return true
        end
    end
    return false
end
```

has_subtour (generic function with 1 method)

```
function mutate_swap(recombinant::AbstractVector; rng::AbstractRNG=Random.default_rng())
    N = length(recombinant)
    a = true
    while a || has_subtour(recombinant)
        i = rand(rng, 1:N)
        j = rand(rng, 1:N)
        v = recombinant[i]
        recombinant[i] = recombinant[j]
        recombinant[j] = v
        a = false
    end
    return recombinant
end
```

"""

Implementation follows "Genetic Algorithm Solution of the TSP Avoiding Special Crossover and
"""

```
function recombine_tsp(v1::T, v2::T; rng::AbstractRNG=Random.default_rng()) where {T <: AbstractVector{<: Integer}}
    N = length(v1)
    i1 = perm_to_inv(v1)
    i2 = perm_to_inv(v2)
    crossover_point = rand(2:(N-1))
    i1c = vcat(i1[1:crossover_point], i2[(crossover_point+1):N])
    i2c = vcat(i2[1:crossover_point], i1[(crossover_point+1):N])
end
```

```

    return inv_to_perm(i1c), inv_to_perm(i2c)
end

```

recombine_tsp

```

x0 = [mod1(i+1, n) for i in 1:n]
function cost(x)
    dist = 0.0
    for i in 1:length(x)
        dist += d[i, x[i]]
    end
    if has_subtour(x)
        dist += 5000
    end
    shopping_cost = sum(minimum(cost_item_city[:, x], dims=2))
    return dist + shopping_cost
end
println("Initial cost: $(cost(x0))")

```

Initial cost: 3504.6196624639356

```

x_sol = Evolutionary.optimize(
    cost,
    x0,
    GA(; populationSize=30000, crossover=recombine_tsp, mutation=mutate_swap, epsilon=0.001,
    Evolutionary.Options(; iterations=10000)
)
println(has_subtour(x_sol.minimizer))
println(x_sol)

```

false

```

* Status: success

* Candidate solution
  Minimizer: [25, 21, 13, ...]
  Minimum:   3063.792207644615
  Iterations: 19

* Found with

```

Algorithm: GA[P=30000,x=0.9,=0.1,=0.001]

* Work counters

Seconds run: 7.6393 (vs limit Inf)
Iterations: 19
f(x) calls: 600006

```
function cost_naive_tpp(v, d, cost_item_city, n, k)
    x = v[1:n]
    c = v[n+1:end]
    dist = 0.0
    for i in 1:n
        dist += d[i, x[i]]
    end
    if has_subtour(x)
        dist += 10000.0
    end
    shopping_cost = 0.0
    for i in 1:k
        shopping_cost += cost_item_city[i,c[i]]
    end
    return dist + shopping_cost
end
```

cost_naive_tpp (generic function with 1 method)

```
function mutate_joint(recombinant::AbstractVector, n, k; rng::AbstractRNG=Random.default_rng)
    route_part = recombinant[1:n]

    a = true
    while a || has_subtour(route_part)
        i = rand(rng, 1:n)
        j = rand(rng, 1:n)
        route_part[i], route_part[j] = route_part[j], route_part[i]
        a = false
    end
    recombinant[1:n] = route_part

    if rand(rng) < 0.3
        item_idx = rand(rng, (n+1):(n+k))
        recombinant[item_idx] = rand(rng, 1:n)
    end
end
```

```

    return recombinant
end

```

mutate_joint (generic function with 1 method)

```

function recombine_joint(v1::T, v2::T, n, k; rng::AbstractRNG=Random.default_rng()) where {T}
    route1, shop1 = v1[1:n], v1[n+1:end]
    route2, shop2 = v2[1:n], v2[n+1:end]

    i1 = perm_to_inv(route1)
    i2 = perm_to_inv(route2)
    cp_route = rand(rng, 2:(n-1))

    i1c = vcat(i1[1:cp_route], i2[(cp_route+1):n])
    i2c = vcat(i2[1:cp_route], i1[(cp_route+1):n])

    new_route1 = inv_to_perm(i1c)
    new_route2 = inv_to_perm(i2c)

    cp_shop = rand(rng, 1:k)
    new_shop1 = vcat(shop1[1:cp_shop], shop2[(cp_shop+1):end])
    new_shop2 = vcat(shop2[1:cp_shop], shop1[(cp_shop+1):end])

    return vcat(new_route1, new_shop1), vcat(new_route2, new_shop2)
end

```

recombine_joint (generic function with 1 method)

```

c0 = rand(1:n, k)
v0 = vcat(x0, c0)

joint_mutation = (v) -> mutate_joint(v, n, k)
joint_crossover = (v1, v2) -> recombine_joint(v1, v2, n, k)
joint_cost = (v) -> cost_naive_tpp(v, d, cost_item_city, n, k)
initial_val = joint_cost(v0)
println("-----")
println("KOSZT PRZED OPTYMALIZACJĄ: ", initial_val)
println("-----")

res = Evolutionary.optimize(

```

```

joint_cost,
v0,
GA(;
    populationSize=100000,
    crossover=joint_crossover,
    mutation=joint_mutation,
    epsilon=0.001,
    crossoverRate=0.9
),
Evolutionary.Options(;
    iterations=10000,
    time_limit=60.0,
    show_trace=false
)
)

v_best = res.minimizer
x_best = v_best[1:n]
c_best = v_best[n+1:end]

println("Status trasy (subtour): ", has_subtour(x_best))
println("Całkowity koszt: ", res.minimum)

```

```

-----
KOSZT PRZED OPTYMALIZACJĄ: 4617.823256961012
-----

Status trasy (subtour): false
Całkowity koszt: 3979.69963840745

```

```

function plot_joint_tour_with_shopping(v_best, X, Y, n, k)
    x_best = v_best[1:n]
    c_best = v_best[n+1:end]

    p = Plots.plot(legend=false, title="Trasa Joint TPP (Decyzje z GA)")

    for i in 1:n
        j = x_best[i]
        Plots.plot!([X[i], X[j]], [Y[i], Y[j]], color=:blue, alpha=0.5)
    end

    z_counts = zeros{Int, n}

```

```

for miasto_zakupu in c_best
    z_counts[miasto_zakupu] += 1
end

Plots.scatter!(X, Y, color=:gray, markersize=2, alpha=0.3)
for i in 1:n
    if z_counts[i] > 0
        msize = 2 + 2 * z_counts[i]
        Plots.scatter!([X[i]], [Y[i]],
            color=:green,
            markersize=msize,
            markerstrokecolor=:white,
            label="")
    end
end

return p
end

```

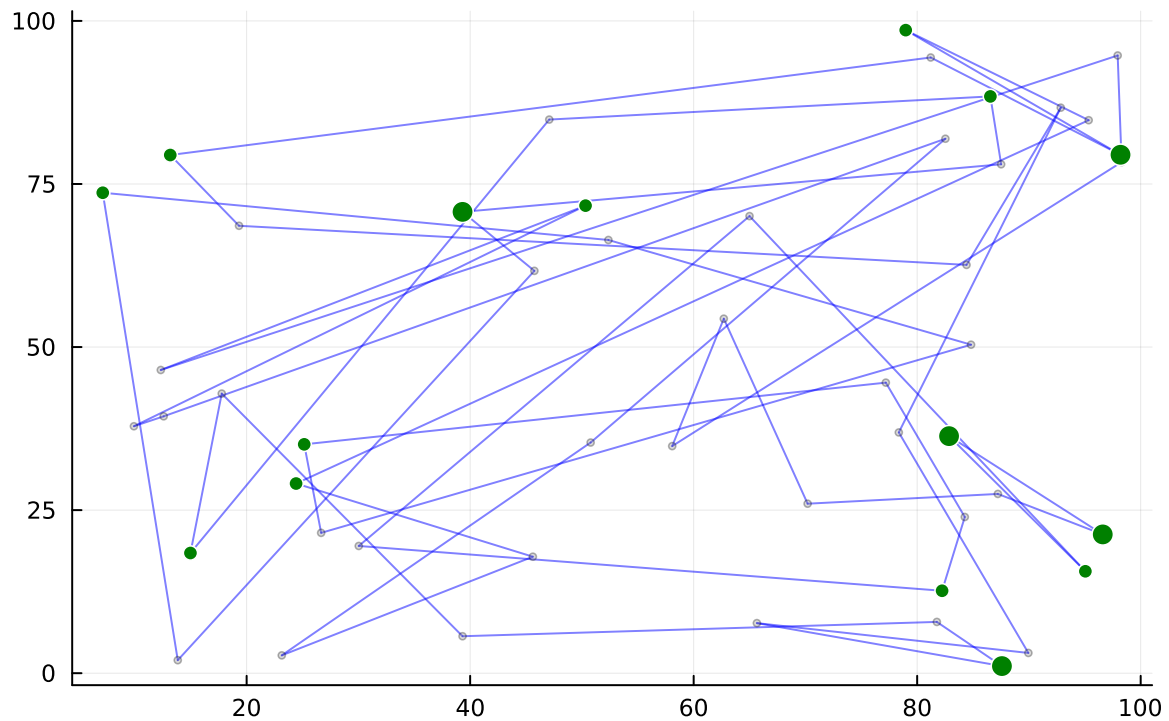
plot_joint_tour_with_shopping (generic function with 1 method)

```

best_individual = res.minimizer
plot_joint_tour_with_shopping(best_individual, X, Y, n, k)

```

Trasa Joint TPP (Decyzje z GA)



```
using Printf
function compare_methods(n,k,seed)
    X, Y, d, cost_item_city = generate_random_tpp(n, k,seed)
    time_iterative = @elapsed begin
        iterative_model = build_tpp_model(d, n, cost_item_city, k)
        optimize!(iterative_model)
        @assert is_solved_and_feasible(iterative_model)
        cycle = subtour(iterative_model[:x])
        while 1 < length(cycle) < n
            S = [(i, j) for (i, j) in Iterators.product(cycle, cycle) if i < j]
            @constraint(
                iterative_model,
                sum(iterative_model[:x][i, j] for (i, j) in S) <= length(cycle) - 1,
            )
            optimize!(iterative_model)
            @assert is_solved_and_feasible(iterative_model)
            cycle = subtour(iterative_model[:x])
        end
    end
    end
    value_iterative = objective_value(iterative_model)
```

```

function cost_local(x)
    dist = 0.0
    for i in 1:length(x)
        dist += d[i, x[i]]
    end
    if has_subtour(x)
        dist += 5000
    end
    shopping_cost = sum(minimum(cost_item_city[:, x], dims=2))
    return dist + shopping_cost
end

time_genetic = @elapsed begin
    x0 = [mod1(i+1, n) for i in 1:n]
    x_sol = Evolutionary.optimize(
        cost_local,
        x0,
        GA(; populationSize=25000, crossover=recombine_tsp, mutation=mutate_swap, epsilon=1e-6),
        Evolutionary.Options(; iterations=10000)
    )
end
value_genetic = Evolutionary.minimum(x_sol)
@printf("%15.2f | %15.4f | %15.2f | %15.4f | %3d | %3d\n", value_iterative, time_iterative, value_genetic, time_genetic, N, K)
end

```

compare_methods (generic function with 1 method)

Porównanie wyników i czasu rozwiązania iteratywnego i genetycznego

```

println("VALUE ITERATIVE | TIME ITERATIVE | VALUE GENETIC | TIME GENETIC | N | K")
compare_methods(50,20,1)
compare_methods(50,20,2)
compare_methods(50,20,3)

compare_methods(50,50,1)
compare_methods(50,50,2)
compare_methods(50,50,3)

```

```
compare_methods(100,100,1)
compare_methods(100,100,2)
compare_methods(100,100,3)
```

VALUE ITERATIVE	TIME ITERATIVE	VALUE GENETIC	TIME GENETIC	N	K
1569.95	0.8366	2987.86	9.6715	50	20
1612.07	0.9880	2817.46	13.8206	50	20
1679.49	0.1349	2858.61	25.9711	50	20
3119.10	0.0736	4587.70	10.2921	50	50
3173.94	1.4611	4434.89	10.8674	50	50
3236.42	0.1997	4509.23	18.9443	50	50
5820.75	376.6011	9343.85	20.9889	100	100
5879.53	74.9094	9031.79	39.6158	100	100
5887.99	24.7007	9412.11	14.3081	100	100

Na 4.0

```
using JuMP, GLPK, Random, Plots, Printf, Evolutionary

function generate_random_tpp4(n, k; random_seed = 1)
    rng = Random.MersenneTwister(random_seed)
    X = 100 .* rand(rng, n)
    Y = 100 .* rand(rng, n)
    d = [hypot(X[i] - X[j], Y[i] - Y[j]) for i in 1:n, j in 1:n]
    cost_item_city = 50 .+ 100 .* rand(rng, k, n)
    item_weight = 1 .+ 9 .* rand(rng, k)
    alpha = 0.01 * (sum(cost_item_city) / length(cost_item_city)) / ((sum(item_weight) / length(item_weight)) / length(cost_item_city))
    return X, Y, d, cost_item_city, item_weight, alpha
end

function solve_jump4(d, cost_item_city)
    model = build_tpp_model(d, size(d, 1), cost_item_city, size(cost_item_city, 1))
    optimize!(model)
    cycle = subtour(model[:x])
    while 1 < length(cycle) < size(d, 1)
        S = [(i, j) for (i, j) in Iterators.product(cycle, cycle) if i < j]
        @constraint(model, sum(model[:x][i, j] for (i, j) in S) <= length(cycle) - 1)
        optimize!(model)
        cycle = subtour(model[:x])
    end
end
```

```

    end
    return model
end

function route_from_x4(x)
    n = size(x, 1)
    neighbors = [Int[] for _ in 1:n]
    for i in 1:n-1, j in i+1:n
        if x[i, j] > 0.5
            push!(neighbors[i], j)
            push!(neighbors[j], i)
        end
    end
    route = Int[1]
    previous_city = 0
    current_city = 1
    while true
        next_city = neighbors[current_city][1] == previous_city ? neighbors[current_city][2]
        if next_city == 1
            break
        end
        push!(route, next_city)
        previous_city, current_city = current_city, next_city
    end
    return route
end

function normalize_route4(route)
    first_city_position = findfirst(==(1), route)
    return vcat(route[first_city_position:end], route[1:first_city_position-1])
end

function cycle_suffix_distances4(route, d)
    n = length(route)
    suffix = zeros(Float64, n)
    total = 0.0
    for position in n:-1:1
        next_position = position == n ? 1 : position + 1
        total += d[route[position], route[next_position]]
        suffix[position] = total
    end
    return suffix
end

```

```

end

function evaluate_tpp4(route, d, cost_item_city, item_weight; alpha = 0.0)
    route = normalize_route4(route)
    suffix = cycle_suffix_distances4(route, d)
    travel_cost = suffix[1]
    shopping_cost = 0.0
    transport_cost = 0.0
    shopping_rows = Tuple{Int, Int, Float64, Float64}[]
    for item in 1:size(cost_item_city, 1)
        best_value = Inf
        best_city = route[1]
        best_price = 0.0
        best_transport = 0.0
        for position in 1:length(route)
            city = route[position]
            price = cost_item_city[item, city]
            transport = alpha * item_weight[item] * suffix[position]
            value = price + transport
            if value < best_value
                best_value = value
                best_city = city
                best_price = price
                best_transport = transport
            end
        end
        shopping_cost += best_price
        transport_cost += best_transport
        push!(shopping_rows, (item, best_city, best_price, best_transport))
    end
    total_cost = travel_cost + shopping_cost + transport_cost
    return total_cost, travel_cost, shopping_cost, transport_cost, shopping_rows
end

function pmx_child4(parent_a::AbstractVector{Int}, parent_b::AbstractVector{Int}; rng = Random.GLOBAL_RNG)
    n = length(parent_a)
    child = zeros{Int, n}
    # choose cut points in [2, n]
    left = rand(rng, 2:(n-1))
    right = rand(rng, left:(n))
    # copy middle segment from parent_a
    child[left:right] = parent_a[left:right]
end

```

```

# fill remaining positions using PMX mapping
for i in 1:n
    if i >= left && i <= right
        continue
    end
    gene = parent_b[i]
    # while gene conflicts with copied segment, map it through parent_a->parent_b
    while gene in child[left:right]
        pos = findfirst==(gene), parent_a)
        gene = parent_b[pos]
    end
    child[i] = gene
end
return normalize_route4(child)
end

function recombine_pmx4(parent_a::AbstractVector{Int}, parent_b::AbstractVector{Int}; rng::AbstractRNG = Random.GLOBAL_RNG)
    return pmx_child4(parent_a, parent_b; rng = rng), pmx_child4(parent_b, parent_a; rng = rng)
end

function solve_ga4(cost_function, start_route, crossover_function; population_size = 60, iterations = 100)
    return Evolutionary.optimize(
        cost_function,
        start_route,
        GA(; populationSize = population_size, crossover = crossover_function, mutation = mutation_function,
        Evolutionary.Options(; iterations = iterations),
    )
end

function plot_route4(X, Y, route; title = "")
    p = plot(title = title, legend = false)
    closed_route = vcat(route, route[1])
    plot!(p, X[closed_route], Y[closed_route], lw = 2)
    scatter!(p, X, Y, ms = 3)
    return p
end

function run_task4_4(n, k, seed)
    X, Y, d, cost_item_city, item_weight, alpha = generate_random_tpp4(n, k; random_seed = seed)

    time_jump = @elapsed jump_model = solve_jump4(d, cost_item_city)
    jump_route = route_from_x4(JuMP.value.(jump_model[:x]))

```

```

jump_base, _, _, _, jump_shopping = evaluate_tpp4(jump_route, d, cost_item_city, item_weight; alpha = 0.0)
jump_weighted, _, _, _, _ = evaluate_tpp4(jump_route, d, cost_item_city, item_weight; alpha = alpha)

start_route = vcat(1, randperm(n - 1) .+ 1)
unweighted_cost(route) = evaluate_tpp4(route, d, cost_item_city, item_weight; alpha = 0.0)
weighted_cost(route) = evaluate_tpp4(route, d, cost_item_city, item_weight; alpha = alpha)

time_ga_inverse_unweighted = @elapsed ga_inverse_unweighted = solve_ga4(unweighted_cost, start_route)
time_ga_inverse_weighted = @elapsed ga_inverse_weighted = solve_ga4(weighted_cost, start_route)
time_ga_pmx_weighted = @elapsed ga_pmx_weighted = solve_ga4(weighted_cost, start_route, 1)

ga_inverse_unweighted_route = normalize_route4(ga_inverse_unweighted.minimizer)
ga_inverse_weighted_route = normalize_route4(ga_inverse_weighted.minimizer)
ga_pmx_weighted_route = normalize_route4(ga_pmx_weighted.minimizer)

ga_inverse_unweighted_base = evaluate_tpp4(ga_inverse_unweighted_route, d, cost_item_city, item_weight; alpha = 0.0)
ga_inverse_unweighted_weighted = evaluate_tpp4(ga_inverse_unweighted_route, d, cost_item_city, item_weight; alpha = alpha)
ga_inverse_weighted_base = evaluate_tpp4(ga_inverse_weighted_route, d, cost_item_city, item_weight; alpha = 0.0)
ga_inverse_weighted_total = evaluate_tpp4(ga_inverse_weighted_route, d, cost_item_city, item_weight; alpha = alpha)
ga_pmx_weighted_base = evaluate_tpp4(ga_pmx_weighted_route, d, cost_item_city, item_weight; alpha = 0.0)
ga_pmx_weighted_total = evaluate_tpp4(ga_pmx_weighted_route, d, cost_item_city, item_weight; alpha = alpha)

@printf("JuMP          | base = %10.2f | weight = %10.2f | time = %7.2fs\n", jump_base, jump_weighted, time_ga_inverse_unweighted, time_ga_inverse_weighted, time_ga_pmx_weighted)
@printf("GA inverse    | base = %10.2f | weight = %10.2f | time = %7.2fs\n", ga_inverse_unweighted_base, ga_inverse_unweighted_weighted, ga_inverse_weighted_base, ga_inverse_weighted_total, ga_pmx_weighted_base, ga_pmx_weighted_total)
@printf("GA inverseW    | base = %10.2f | weight = %10.2f | time = %7.2fs\n", ga_inverse_weighted_base, ga_inverse_weighted_total, ga_pmx_weighted_base, ga_pmx_weighted_total)
@printf("GA PMX        | base = %10.2f | weight = %10.2f | time = %7.2fs\n", ga_pmx_weighted_base, ga_pmx_weighted_total)

println("\nZakupy JuMP:")
for (item, city, price, transport) in jump_shopping
    println("item ", item, " -> city ", city, " (price=", round(price, digits = 2), ", transport=", transport)
end

println("\nTrasa JuMP: ", jump_route)
println("Trasa GA PMX: ", ga_pmx_weighted_route)

println("\nZakupy GA PMX:")
for (item, city, price, transport) in evaluate_tpp4(ga_pmx_weighted_route, d, cost_item_city, item_weight; alpha = alpha)
    println("item ", item, " -> city ", city, " (price=", round(price, digits = 2), ", transport=", transport)
end

return plot_route4(X, Y, ga_pmx_weighted_route; title = "TPP 4.0 - GA PMX")
end

```

```
# run_task4_4(50, 20, 1)
```

run_task4_4 (generic function with 1 method)

```
run_task4_4(50, 20, 1)
```

JuMP	base =	1569.95	weight =	1603.37	time =	1.30s
GA inverse	base =	3060.88	weight =	3150.75	time =	0.03s
GA inverseW	base =	3188.90	weight =	3275.89	time =	0.03s
GA PMX	base =	3298.06	weight =	3386.47	time =	0.07s

Zakupy JuMP:

item 1 -> city 46 (price=56.93, transport=0.0)
item 2 -> city 33 (price=50.98, transport=0.0)
item 3 -> city 41 (price=51.65, transport=0.0)
item 4 -> city 25 (price=50.39, transport=0.0)
item 5 -> city 13 (price=50.33, transport=0.0)
item 6 -> city 16 (price=50.74, transport=0.0)
item 7 -> city 16 (price=50.5, transport=0.0)
item 8 -> city 16 (price=51.12, transport=0.0)
item 9 -> city 6 (price=51.11, transport=0.0)
item 10 -> city 19 (price=50.1, transport=0.0)
item 11 -> city 2 (price=50.8, transport=0.0)
item 12 -> city 4 (price=50.72, transport=0.0)
item 13 -> city 4 (price=50.96, transport=0.0)
item 14 -> city 40 (price=50.24, transport=0.0)
item 15 -> city 25 (price=52.44, transport=0.0)
item 16 -> city 13 (price=50.89, transport=0.0)
item 17 -> city 22 (price=50.13, transport=0.0)
item 18 -> city 7 (price=52.88, transport=0.0)
item 19 -> city 14 (price=51.06, transport=0.0)
item 20 -> city 6 (price=50.19, transport=0.0)

Trasa JuMP: [1, 19, 11, 26, 35, 30, 34, 20, 9, 31, 43, 27, 6, 18, 2, 29, 13, 38, 4, 28, 44, 1]

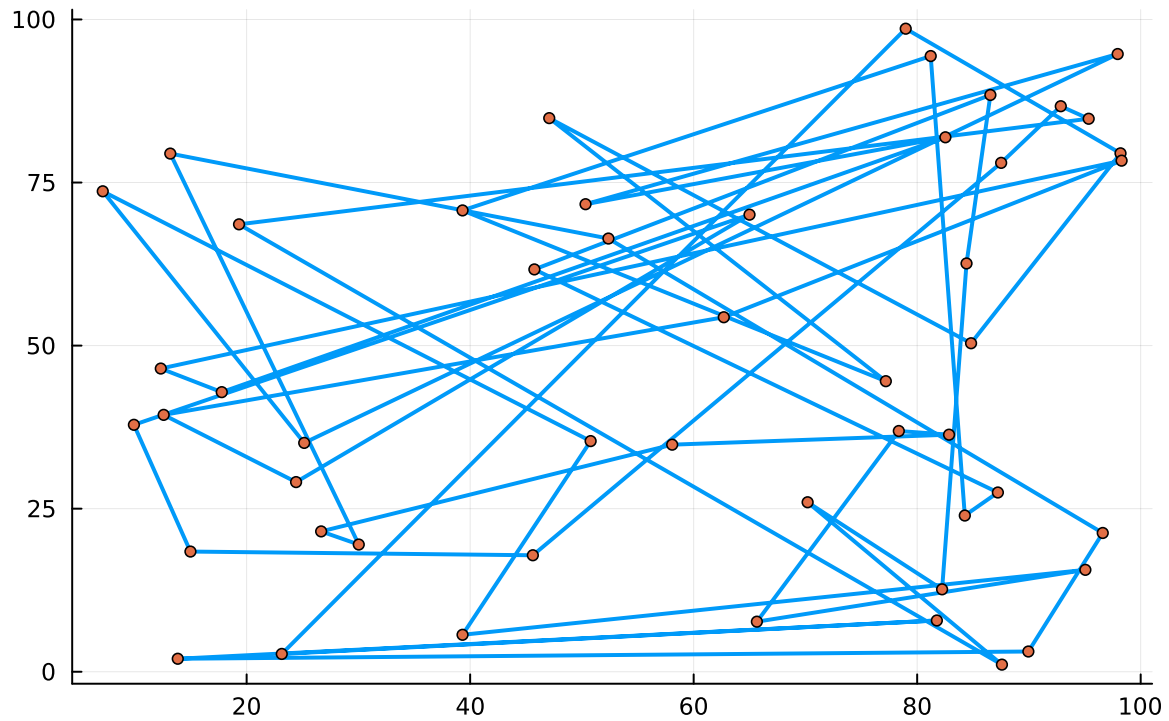
Trasa GA PMX: [1, 20, 27, 39, 50, 5, 49, 4, 2, 13, 12, 24, 42, 40, 37, 33, 10, 36, 41, 21, 1]

Zakupy GA PMX:

item 1 -> city 46 (price=56.93, transport=0.62)
item 2 -> city 21 (price=51.28, transport=3.3)
item 3 -> city 48 (price=58.17, transport=4.16)

item 4 -> city 25 (price=50.39, transport=2.29)
 item 5 -> city 25 (price=51.0, transport=4.4)
 item 6 -> city 16 (price=50.74, transport=1.27)
 item 7 -> city 16 (price=50.5, transport=2.11)
 item 8 -> city 16 (price=51.12, transport=1.22)
 item 9 -> city 6 (price=51.11, transport=2.03)
 item 10 -> city 19 (price=50.1, transport=0.36)
 item 11 -> city 34 (price=51.5, transport=2.12)
 item 12 -> city 4 (price=50.72, transport=6.3)
 item 13 -> city 9 (price=53.11, transport=11.47)
 item 14 -> city 16 (price=53.08, transport=2.69)
 item 15 -> city 25 (price=52.44, transport=4.32)
 item 16 -> city 8 (price=54.92, transport=6.64)
 item 17 -> city 22 (price=50.13, transport=2.42)
 item 18 -> city 22 (price=53.94, transport=2.1)
 item 19 -> city 14 (price=51.06, transport=4.41)
 item 20 -> city 16 (price=53.74, transport=2.4)

TPP 4.0 - GA PMX



Zad na 5.0

```
# Generowanie danych dla zadania 5.0

function generate_random_tpp5(n, k; random_seed = 42)
    rng = Random.MersenneTwister(random_seed)
    X = 100 .* rand(rng, n)
    Y = 100 .* rand(rng, n)
    d = [hypot(X[i] - X[j], Y[i] - Y[j]) for i in 1:n, j in 1:n]

    base_cost = 50 .+ 100 .* rand(rng, k, n)
    cost_item_city = copy(base_cost)

    for item in 1:k
        cheap_city = rand(rng, 1:n)
        cost_item_city[item, cheap_city] = max(10.0, base_cost[item, cheap_city] - 40.0)

        expensive_count = max(2, n ÷ 3)
        expensive_cities = randperm(rng, n)[1:expensive_count]

        for c in expensive_cities
            if c != cheap_city
                cost_item_city[item, c] = base_cost[item, c] + 500.0
            end
        end
    end

    item_weight = 1 .+ 9 .* rand(rng, k)

    alpha = 0.01 * (sum(cost_item_city) / length(cost_item_city)) /
        ((sum(item_weight) / length(item_weight)) * maximum(d))

    return X, Y, d, cost_item_city, item_weight, alpha
end

n5, k5 = 10, 5
X5, Y5, d5, c5, w5, alpha5 = generate_random_tpp5(n5, k5; random_seed = 42)
```

([71.08238673434464, 6.44852510983267, 47.784264106691495, 17.770930557953246, 5.35198874829

```

# Model MIP z opcjonalnym odwiedzaniem miast
# Miasto jest odwiedzane tylko wtedy, gdy kupujemy w nim produkt.
function build_optional_tpp_model(d, n, cost_item_city, m)
    model = Model(GLPK.Optimizer)

    @variable(model, x[1:n, 1:n], Bin, Symmetric)
    @variable(model, y[1:n], Bin)
    @variable(model, z[1:m, 1:n], Bin)

    @objective(model, Min, (sum(d .* x) / 2) + sum(cost_item_city .* z))

    @constraint(model, [p in 1:m], sum(z[p, :]) == 1)
    @constraint(model, [p in 1:m, i in 1:n], z[p, i] <= y[i])

    @constraint(model, [i in 1:n], sum(x[i, :]) == 2 * y[i])
    @constraint(model, [i in 1:n], x[i, i] == 0)

    @constraint(model, sum(y) >= 3)

    return model
end

```

build_optional_tpp_model (generic function with 1 method)

```

function solve_optional_tpp(d, n, cost_item_city, k)
    model = build_optional_tpp_model(d, n, cost_item_city, k)

    println("Start optymalizacji MIP...")
    optimize!(model)

    println("Pierwsze rozwiązanie znalezione")

    iter = 0
    function find_subtour(x_val)
        n = size(x_val, 1)
        visited = falses(n)

        for start in 1:n
            if !visited[start]
                cycle = Int[]
                stack = [start]

```

```

        while !isempty(stack)
            i = pop!(stack)

            if !visited[i]
                push!(cycle, i)
                visited[i] = true

                neighbors = findall(x_val[i, :] .> 0.5)
                append!(stack, neighbors)
            end
        end

        if length(cycle) > 1
            return cycle
        end
    end
end

return Int[]
end

while true
    iter += 1

    println("Iteracja: ", iter)

    x_val = JuMP.value.(model[:x])
    y_val = JuMP.value.(model[:y])

    cycle = find_subtour(x_val)

    println("Odwiedzone miasta: ", sum(y_val .> 0.5))
    println("Znaleziony cykl: ", cycle)

    if isempty(cycle)
        println("Brak podcykli - koniec")
        break
    end

    println("Dodaję constraint dla cyklu długości ", length(cycle))

    @constraint(model,

```

```

        sum(model[:x][i,j] for i in cycle, j in cycle if i < j) <= length(cycle)-1
    )

    println("Ponowne optymalizowanie...")

    optimize!(model)

    println("Zakończono iterację")
end

println("Rozwiązanie końcowe znalezione")

println("Liczba odwiedzonych miast: ", sum(JuMP.value.(model[:y]) .> 0.5))
println("Koszt: ", objective_value(model))

return model
end

```

solve_optional_tpp (generic function with 1 method)

```

opt_model5 = solve_optional_tpp(d5, n5, c5, k5)

println("Koszt MIP: ", objective_value(opt_model5))
println("Odwiedzone miasta: ", sum(value.(opt_model5[:y]) .> 0.5))

```

```

Start optymalizacji MIP...
Pierwsze rozwiązanie znalezione
Iteracja: 1
Odwiedzone miasta: 3
Znaleziony cykl: [4, 9, 8]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 2
Odwiedzone miasta: 4
Znaleziony cykl: [2, 9, 4, 8]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 3
Odwiedzone miasta: 3

```

Znaleziony cykl: [2, 9, 8]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 4
Odwiedzone miasta: 3
Znaleziony cykl: [2, 8, 4]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 5
Odwiedzone miasta: 3
Znaleziony cykl: [4, 10, 8]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 6
Odwiedzone miasta: 4
Znaleziony cykl: [4, 9, 7, 8]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 7
Odwiedzone miasta: 4
Znaleziony cykl: [4, 8, 10, 7]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 8
Odwiedzone miasta: 4
Znaleziony cykl: [2, 8, 10, 4]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 9
Odwiedzone miasta: 4
Znaleziony cykl: [4, 9, 10, 8]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 10
Odwiedzone miasta: 5
Znaleziony cykl: [2, 8, 7, 9, 4]

Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 11
Odwiedzone miasta: 4
Znaleziony cykl: [2, 9, 7, 8]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 12
Odwiedzone miasta: 3
Znaleziony cykl: [2, 10, 8]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 13
Odwiedzone miasta: 5
Znaleziony cykl: [4, 9, 7, 10, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 14
Odwiedzone miasta: 5
Znaleziony cykl: [2, 8, 10, 7, 4]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 15
Odwiedzone miasta: 4
Znaleziony cykl: [2, 8, 10, 7]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 16
Odwiedzone miasta: 5
Znaleziony cykl: [2, 8, 10, 9, 4]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 17
Odwiedzone miasta: 4
Znaleziony cykl: [2, 9, 10, 8]
Dodaję constraint dla cyklu długości 4

Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 18
Odwiedzone miasta: 3
Znaleziony cykl: [3, 10, 8]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 19
Odwiedzone miasta: 3
Znaleziony cykl: [7, 9, 8]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 20
Odwiedzone miasta: 6
Znaleziony cykl: [2, 8, 10, 7, 9, 4]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 21
Odwiedzone miasta: 5
Znaleziony cykl: [2, 9, 7, 10, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 22
Odwiedzone miasta: 4
Znaleziony cykl: [4, 10, 6, 8]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 23
Odwiedzone miasta: 4
Znaleziony cykl: [3, 10, 4, 8]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 24
Odwiedzone miasta: 4
Znaleziony cykl: [3, 10, 8, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...

Zakończono iterację
Iteracja: 25
Odwiedzone miasta: 3
Znaleziony cykl: [4, 8, 7]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 26
Odwiedzone miasta: 4
Znaleziony cykl: [3, 10, 7, 8]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 27
Odwiedzone miasta: 4
Znaleziony cykl: [4, 9, 6, 8]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 28
Odwiedzone miasta: 5
Znaleziony cykl: [4, 8, 6, 10, 7]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 29
Odwiedzone miasta: 5
Znaleziony cykl: [3, 10, 7, 4, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 30
Odwiedzone miasta: 5
Znaleziony cykl: [3, 10, 4, 8, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 31
Odwiedzone miasta: 3
Znaleziony cykl: [8, 10, 9]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację

Iteracja: 32
Odwiedzone miasta: 4
Znaleziony cykl: [7, 10, 8, 9]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 33
Odwiedzone miasta: 5
Znaleziony cykl: [3, 10, 7, 8, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 34
Odwiedzone miasta: 5
Znaleziony cykl: [2, 8, 6, 10, 4]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 35
Odwiedzone miasta: 3
Znaleziony cykl: [2, 9, 4]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 36
Odwiedzone miasta: 3
Znaleziony cykl: [3, 10, 6]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 37
Odwiedzone miasta: 6
Znaleziony cykl: [3, 10, 7, 4, 8, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 38
Odwiedzone miasta: 5
Znaleziony cykl: [4, 9, 10, 6, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 39

Odwiedzone miasta: 5
Znaleziony cykl: [3, 10, 9, 4, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 40
Odwiedzone miasta: 5
Znaleziony cykl: [2, 8, 3, 10, 4]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 41
Odwiedzone miasta: 4
Znaleziony cykl: [2, 8, 10, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 42
Odwiedzone miasta: 5
Znaleziony cykl: [2, 9, 4, 6, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 43
Odwiedzone miasta: 4
Znaleziony cykl: [2, 8, 7, 4]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 44
Odwiedzone miasta: 4
Znaleziony cykl: [2, 10, 3, 8]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 45
Odwiedzone miasta: 6
Znaleziony cykl: [4, 9, 7, 10, 6, 8]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 46
Odwiedzone miasta: 4

Znaleziony cykl: [3, 9, 4, 8]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 47
Odwiedzone miasta: 6
Znaleziony cykl: [3, 10, 7, 9, 4, 8]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 48
Odwiedzone miasta: 4
Znaleziony cykl: [2, 9, 6, 8]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 49
Odwiedzone miasta: 4
Znaleziony cykl: [3, 10, 9, 8]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 50
Odwiedzone miasta: 6
Znaleziony cykl: [2, 8, 6, 10, 7, 4]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 51
Odwiedzone miasta: 5
Znaleziony cykl: [4, 9, 7, 6, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 52
Odwiedzone miasta: 6
Znaleziony cykl: [2, 8, 3, 10, 7, 4]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 53
Odwiedzone miasta: 3
Znaleziony cykl: [2, 8, 7]

Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 54
Odwiedzone miasta: 3
Znaleziony cykl: [3, 9, 8]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 55
Odwiedzone miasta: 6
Znaleziony cykl: [3, 10, 9, 4, 8, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 56
Odwiedzone miasta: 5
Znaleziony cykl: [3, 10, 7, 9, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 57
Odwiedzone miasta: 6
Znaleziony cykl: [2, 8, 6, 3, 10, 4]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 58
Odwiedzone miasta: 5
Znaleziony cykl: [2, 8, 6, 10, 7]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 59
Odwiedzone miasta: 5
Znaleziony cykl: [3, 8, 4, 9, 7]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 60
Odwiedzone miasta: 5
Znaleziony cykl: [2, 8, 10, 3, 6]
Dodaję constraint dla cyklu długości 5

Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 61
Odwiedzone miasta: 5
Znaleziony cykl: [2, 8, 3, 10, 7]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 62
Odwiedzone miasta: 5
Znaleziony cykl: [3, 9, 4, 8, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 63
Odwiedzone miasta: 7
Znaleziony cykl: [3, 10, 7, 9, 4, 8, 6]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 64
Odwiedzone miasta: 5
Znaleziony cykl: [3, 10, 9, 8, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 65
Odwiedzone miasta: 6
Znaleziony cykl: [2, 8, 6, 10, 9, 4]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 66
Odwiedzone miasta: 5
Znaleziony cykl: [2, 9, 10, 6, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 67
Odwiedzone miasta: 4
Znaleziony cykl: [3, 8, 9, 7]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...

Zakończono iterację
Iteracja: 68
Odwiedzone miasta: 7
Znaleziony cykl: [2, 8, 6, 3, 10, 7, 4]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 69
Odwiedzone miasta: 6
Znaleziony cykl: [2, 8, 3, 10, 9, 4]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 70
Odwiedzone miasta: 5
Znaleziony cykl: [2, 9, 10, 3, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 71
Odwiedzone miasta: 4
Znaleziony cykl: [3, 9, 8, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 72
Odwiedzone miasta: 6
Znaleziony cykl: [3, 10, 7, 9, 8, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 73
Odwiedzone miasta: 7
Znaleziony cykl: [2, 8, 6, 10, 7, 9, 4]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 74
Odwiedzone miasta: 6
Znaleziony cykl: [2, 9, 7, 10, 6, 8]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację

Iteracja: 75
Odwiedzone miasta: 6
Znaleziony cykl: [3, 7, 9, 4, 8, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 76
Odwiedzone miasta: 5
Znaleziony cykl: [2, 9, 4, 3, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 77
Odwiedzone miasta: 7
Znaleziony cykl: [2, 8, 3, 10, 7, 9, 4]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 78
Odwiedzone miasta: 4
Znaleziony cykl: [2, 9, 3, 8]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 79
Odwiedzone miasta: 6
Znaleziony cykl: [2, 9, 7, 10, 3, 8]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 80
Odwiedzone miasta: 6
Znaleziony cykl: [2, 8, 6, 3, 10, 7]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 81
Odwiedzone miasta: 3
Znaleziony cykl: [6, 9, 8]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 82

Odwiedzone miasta: 6
Znaleziony cykl: [2, 8, 6, 7, 9, 4]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 83
Odwiedzone miasta: 5
Znaleziony cykl: [2, 9, 7, 6, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 84
Odwiedzone miasta: 3
Znaleziony cykl: [4, 8, 6]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 85
Odwiedzone miasta: 5
Znaleziony cykl: [3, 7, 9, 8, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 86
Odwiedzone miasta: 7
Znaleziony cykl: [2, 8, 6, 3, 10, 9, 4]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 87
Odwiedzone miasta: 6
Znaleziony cykl: [2, 9, 10, 3, 6, 8]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 88
Odwiedzone miasta: 6
Znaleziony cykl: [2, 8, 3, 7, 9, 4]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 89
Odwiedzone miasta: 5

Znaleziony cykl: [2, 9, 7, 3, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 90
Odwiedzone miasta: 6
Znaleziony cykl: [2, 9, 4, 3, 6, 8]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 91
Odwiedzone miasta: 8
Znaleziony cykl: [2, 8, 6, 3, 10, 7, 9, 4]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 92
Odwiedzone miasta: 5
Znaleziony cykl: [2, 9, 3, 6, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 93
Odwiedzone miasta: 7
Znaleziony cykl: [2, 9, 7, 10, 3, 6, 8]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 94
Odwiedzone miasta: 4
Znaleziony cykl: [3, 10, 7, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 95
Odwiedzone miasta: 4
Znaleziony cykl: [6, 10, 9, 8]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 96
Odwiedzone miasta: 3
Znaleziony cykl: [2, 8, 6]

Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 97
Odwiedzone miasta: 7
Znaleziony cykl: [2, 8, 6, 3, 7, 9, 4]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 98
Odwiedzone miasta: 5
Znaleziony cykl: [6, 10, 7, 9, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 99
Odwiedzone miasta: 6
Znaleziony cykl: [2, 9, 7, 3, 6, 8]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 100
Odwiedzone miasta: 3
Znaleziony cykl: [7, 10, 8]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 101
Odwiedzone miasta: 4
Znaleziony cykl: [2, 8, 6, 4]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 102
Odwiedzone miasta: 4
Znaleziony cykl: [6, 8, 9, 7]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 103
Odwiedzone miasta: 3
Znaleziony cykl: [3, 8, 4]
Dodaję constraint dla cyklu długości 3

Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 104
Odwiedzone miasta: 3
Znaleziony cykl: [3, 8, 6]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 105
Odwiedzone miasta: 3
Znaleziony cykl: [3, 10, 7]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 106
Odwiedzone miasta: 4
Znaleziony cykl: [4, 8, 6, 7]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 107
Odwiedzone miasta: 4
Znaleziony cykl: [3, 6, 8, 4]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 108
Odwiedzone miasta: 4
Znaleziony cykl: [1, 10, 8, 3]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 109
Odwiedzone miasta: 3
Znaleziony cykl: [3, 8, 7]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 110
Odwiedzone miasta: 5
Znaleziony cykl: [4, 10, 6, 5, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...

Zakończono iterację
Iteracja: 111
Odwiedzone miasta: 4
Znaleziony cykl: [4, 9, 5, 8]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 112
Odwiedzone miasta: 4
Znaleziony cykl: [3, 8, 4, 7]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 113
Odwiedzone miasta: 5
Znaleziony cykl: [3, 10, 8, 5, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 114
Odwiedzone miasta: 5
Znaleziony cykl: [1, 10, 4, 8, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 115
Odwiedzone miasta: 4
Znaleziony cykl: [1, 10, 4, 8]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 116
Odwiedzone miasta: 4
Znaleziony cykl: [2, 8, 3, 4]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 117
Odwiedzone miasta: 5
Znaleziony cykl: [1, 6, 8, 10, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację

Iteracja: 118
Odwiedzone miasta: 5
Znaleziony cykl: [2, 9, 4, 8, 5]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 119
Odwiedzone miasta: 3
Znaleziony cykl: [2, 8, 3]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 120
Odwiedzone miasta: 5
Znaleziony cykl: [4, 9, 6, 5, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 121
Odwiedzone miasta: 6
Znaleziony cykl: [4, 8, 5, 6, 10, 7]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 122
Odwiedzone miasta: 4
Znaleziony cykl: [4, 10, 5, 8]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 123
Odwiedzone miasta: 4
Znaleziony cykl: [3, 7, 8, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 124
Odwiedzone miasta: 5
Znaleziony cykl: [1, 10, 4, 8, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 125

Odwiedzone miasta: 5
Znaleziony cykl: [1, 10, 7, 8, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 126
Odwiedzone miasta: 6
Znaleziony cykl: [3, 10, 4, 8, 5, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 127
Odwiedzone miasta: 5
Znaleziony cykl: [3, 7, 4, 8, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 128
Odwiedzone miasta: 4
Znaleziony cykl: [3, 10, 8, 5]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 129
Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 7, 4, 8, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 130
Odwiedzone miasta: 4
Znaleziony cykl: [2, 9, 8, 5]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 131
Odwiedzone miasta: 6
Znaleziony cykl: [1, 6, 8, 4, 10, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 132
Odwiedzone miasta: 5

Znaleziony cykl: [1, 10, 7, 4, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 133
Odwiedzone miasta: 5
Znaleziony cykl: [2, 8, 6, 7, 4]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 134
Odwiedzone miasta: 4
Znaleziony cykl: [2, 9, 7, 4]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 135
Odwiedzone miasta: 5
Znaleziony cykl: [2, 8, 10, 6, 5]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 136
Odwiedzone miasta: 4
Znaleziony cykl: [3, 10, 4, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 137
Odwiedzone miasta: 4
Znaleziony cykl: [2, 8, 3, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 138
Odwiedzone miasta: 6
Znaleziony cykl: [3, 10, 7, 8, 5, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 139
Odwiedzone miasta: 5
Znaleziony cykl: [4, 8, 5, 10, 7]

Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 140
Odwiedzone miasta: 3
Znaleziony cykl: [6, 10, 8]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 141
Odwiedzone miasta: 5
Znaleziony cykl: [2, 8, 6, 3, 4]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 142
Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 7, 4, 8, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 143
Odwiedzone miasta: 6
Znaleziony cykl: [2, 8, 5, 6, 10, 4]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 144
Odwiedzone miasta: 6
Znaleziony cykl: [2, 9, 4, 8, 6, 5]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 145
Odwiedzone miasta: 5
Znaleziony cykl: [3, 10, 4, 8, 5]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 146
Odwiedzone miasta: 6
Znaleziony cykl: [1, 6, 8, 7, 10, 3]
Dodaję constraint dla cyklu długości 6

Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 147
Odwiedzone miasta: 7
Znaleziony cykl: [3, 10, 7, 4, 8, 5, 6]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 148
Odwiedzone miasta: 6
Znaleziony cykl: [4, 9, 10, 6, 5, 8]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 149
Odwiedzone miasta: 3
Znaleziony cykl: [2, 10, 4]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 150
Odwiedzone miasta: 5
Znaleziony cykl: [2, 8, 3, 7, 4]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 151
Odwiedzone miasta: 4
Znaleziony cykl: [2, 8, 6, 7]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 152
Odwiedzone miasta: 4
Znaleziony cykl: [1, 6, 10, 3]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 153
Odwiedzone miasta: 7
Znaleziony cykl: [1, 6, 8, 4, 7, 10, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...

Zakończono iterację
Iteracja: 154
Odwiedzone miasta: 5
Znaleziony cykl: [3, 10, 7, 4, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 155
Odwiedzone miasta: 7
Znaleziony cykl: [4, 9, 7, 10, 6, 5, 8]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 156
Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 9, 4, 8, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 157
Odwiedzone miasta: 5
Znaleziony cykl: [1, 10, 4, 2, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 158
Odwiedzone miasta: 4
Znaleziony cykl: [2, 8, 10, 5]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 159
Odwiedzone miasta: 5
Znaleziony cykl: [3, 10, 7, 8, 5]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 160
Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 4, 2, 8, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację

Iteracja: 161
Odwiedzone miasta: 5
Znaleziony cykl: [2, 9, 8, 6, 5]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 162
Odwiedzone miasta: 5
Znaleziony cykl: [1, 10, 9, 4, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 163
Odwiedzone miasta: 7
Znaleziony cykl: [2, 8, 5, 6, 10, 7, 4]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 164
Odwiedzone miasta: 4
Znaleziony cykl: [2, 8, 3, 7]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 165
Odwiedzone miasta: 5
Znaleziony cykl: [2, 8, 5, 10, 4]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 166
Odwiedzone miasta: 6
Znaleziony cykl: [3, 10, 7, 4, 8, 5]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 167
Odwiedzone miasta: 4
Znaleziony cykl: [1, 10, 2, 8]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 168

Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 4, 2, 8, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 169
Odwiedzone miasta: 5
Znaleziony cykl: [1, 10, 2, 8, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 170
Odwiedzone miasta: 6
Znaleziony cykl: [2, 8, 7, 10, 6, 5]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 171
Odwiedzone miasta: 5
Znaleziony cykl: [4, 9, 10, 5, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 172
Odwiedzone miasta: 6
Znaleziony cykl: [4, 9, 7, 6, 5, 8]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 173
Odwiedzone miasta: 6
Znaleziony cykl: [2, 8, 10, 3, 6, 5]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 174
Odwiedzone miasta: 7
Znaleziony cykl: [1, 10, 7, 9, 4, 8, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 175
Odwiedzone miasta: 4

Znaleziony cykl: [2, 10, 7, 4]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 176
Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 9, 4, 8, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 177
Odwiedzone miasta: 3
Znaleziony cykl: [2, 9, 7]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 178
Odwiedzone miasta: 4
Znaleziony cykl: [6, 10, 7, 8]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 179
Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 7, 9, 4, 8]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 180
Odwiedzone miasta: 5
Znaleziony cykl: [1, 10, 8, 2, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 181
Odwiedzone miasta: 5
Znaleziony cykl: [1, 10, 9, 8, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 182
Odwiedzone miasta: 7
Znaleziony cykl: [3, 10, 9, 4, 8, 5, 6]

Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 183
Odwiedzone miasta: 7
Znaleziony cykl: [2, 8, 5, 6, 3, 10, 4]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 184
Odwiedzone miasta: 3
Znaleziony cykl: [4, 10, 6]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 185
Odwiedzone miasta: 6
Znaleziony cykl: [2, 8, 6, 3, 7, 4]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 186
Odwiedzone miasta: 5
Znaleziony cykl: [1, 9, 4, 8, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 187
Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 7, 4, 2, 8]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 188
Odwiedzone miasta: 7
Znaleziony cykl: [1, 10, 7, 4, 2, 8, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 189
Odwiedzone miasta: 6
Znaleziony cykl: [4, 9, 7, 10, 5, 8]
Dodaję constraint dla cyklu długości 6

Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 190
Odwiedzone miasta: 4
Znaleziony cykl: [1, 9, 4, 8]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 191
Odwiedzone miasta: 7
Znaleziony cykl: [1, 6, 8, 4, 9, 10, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 192
Odwiedzone miasta: 7
Znaleziony cykl: [1, 10, 7, 9, 4, 8, 6]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 193
Odwiedzone miasta: 7
Znaleziony cykl: [1, 6, 8, 2, 4, 10, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 194
Odwiedzone miasta: 6
Znaleziony cykl: [1, 7, 9, 4, 8, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 195
Odwiedzone miasta: 5
Znaleziony cykl: [4, 9, 7, 5, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 196
Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 7, 9, 8, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...

Zakończono iterację
Iteracja: 197
Odwiedzone miasta: 6
Znaleziony cykl: [3, 9, 4, 8, 5, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 198
Odwiedzone miasta: 3
Znaleziony cykl: [1, 10, 3]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 199
Odwiedzone miasta: 6
Znaleziony cykl: [2, 8, 5, 10, 7, 4]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 200
Odwiedzone miasta: 8
Znaleziony cykl: [3, 10, 7, 9, 4, 8, 5, 6]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 201
Odwiedzone miasta: 5
Znaleziony cykl: [3, 10, 9, 4, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 202
Odwiedzone miasta: 5
Znaleziony cykl: [1, 8, 4, 9, 7]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 203
Odwiedzone miasta: 7
Znaleziony cykl: [1, 10, 7, 4, 2, 8, 6]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację

Iteracja: 204
Odwiedzone miasta: 6
Znaleziony cykl: [1, 6, 2, 8, 10, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 205
Odwiedzone miasta: 5
Znaleziony cykl: [2, 8, 10, 3, 5]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 206
Odwiedzone miasta: 5
Znaleziony cykl: [1, 9, 4, 8, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 207
Odwiedzone miasta: 5
Znaleziony cykl: [2, 8, 7, 10, 5]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 208
Odwiedzone miasta: 6
Znaleziony cykl: [3, 10, 9, 8, 5, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 209
Odwiedzone miasta: 5
Znaleziony cykl: [2, 8, 6, 3, 7]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 210
Odwiedzone miasta: 4
Znaleziony cykl: [2, 9, 10, 4]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 211

Odwiedzone miasta: 6
Znaleziony cykl: [1, 6, 8, 4, 9, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 212
Odwiedzone miasta: 5
Znaleziony cykl: [2, 6, 3, 10, 4]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 213
Odwiedzone miasta: 7
Znaleziony cykl: [2, 8, 5, 6, 10, 9, 4]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 214
Odwiedzone miasta: 8
Znaleziony cykl: [1, 6, 8, 4, 9, 7, 10, 3]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 215
Odwiedzone miasta: 5
Znaleziony cykl: [1, 10, 7, 2, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 216
Odwiedzone miasta: 4
Znaleziony cykl: [1, 9, 8, 3]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 217
Odwiedzone miasta: 6
Znaleziony cykl: [2, 9, 10, 6, 5, 8]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 218
Odwiedzone miasta: 6

Znaleziony cykl: [1, 10, 7, 2, 8, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 219
Odwiedzone miasta: 3
Znaleziony cykl: [4, 9, 6]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 220
Odwiedzone miasta: 6
Znaleziony cykl: [3, 10, 9, 4, 8, 5]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 221
Odwiedzone miasta: 4
Znaleziony cykl: [3, 10, 6, 5]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 222
Odwiedzone miasta: 6
Znaleziony cykl: [2, 8, 5, 3, 10, 4]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 223
Odwiedzone miasta: 8
Znaleziony cykl: [2, 8, 5, 6, 3, 10, 7, 4]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 224
Odwiedzone miasta: 3
Znaleziony cykl: [4, 8, 5]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 225
Odwiedzone miasta: 6
Znaleziony cykl: [1, 7, 9, 4, 8, 6]

Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 226
Odwiedzone miasta: 4
Znaleziony cykl: [4, 7, 10, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 227
Odwiedzone miasta: 3
Znaleziony cykl: [5, 9, 8]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 228
Odwiedzone miasta: 6
Znaleziony cykl: [1, 6, 8, 9, 10, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 229
Odwiedzone miasta: 4
Znaleziony cykl: [3, 9, 4, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 230
Odwiedzone miasta: 6
Znaleziony cykl: [3, 10, 7, 9, 4, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 231
Odwiedzone miasta: 5
Znaleziony cykl: [1, 7, 9, 8, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 232
Odwiedzone miasta: 7
Znaleziony cykl: [2, 8, 7, 10, 3, 6, 5]
Dodaję constraint dla cyklu długości 7

Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 233
Odwiedzone miasta: 5
Znaleziony cykl: [3, 9, 8, 5, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 234
Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 7, 2, 8, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 235
Odwiedzone miasta: 7
Znaleziony cykl: [3, 10, 7, 9, 8, 5, 6]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 236
Odwiedzone miasta: 5
Znaleziony cykl: [2, 9, 7, 10, 4]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 237
Odwiedzone miasta: 8
Znaleziony cykl: [1, 6, 8, 2, 4, 7, 10, 3]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 238
Odwiedzone miasta: 4
Znaleziony cykl: [2, 6, 10, 4]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 239
Odwiedzone miasta: 8
Znaleziony cykl: [2, 8, 5, 6, 10, 7, 9, 4]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...

Zakończono iterację
Iteracja: 240
Odwiedzone miasta: 5
Znaleziony cykl: [3, 9, 4, 8, 5]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 241
Odwiedzone miasta: 7
Znaleziony cykl: [2, 9, 7, 10, 6, 5, 8]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 242
Odwiedzone miasta: 7
Znaleziony cykl: [3, 7, 9, 4, 8, 5, 6]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 243
Odwiedzone miasta: 7
Znaleziony cykl: [3, 10, 7, 9, 4, 8, 5]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 244
Odwiedzone miasta: 3
Znaleziony cykl: [1, 10, 8]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 245
Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 9, 4, 2, 8]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 246
Odwiedzone miasta: 5
Znaleziony cykl: [1, 6, 8, 9, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację

Iteracja: 247
Odwiedzone miasta: 7
Znaleziony cykl: [1, 10, 9, 4, 2, 8, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 248
Odwiedzone miasta: 7
Znaleziony cykl: [1, 6, 8, 9, 7, 10, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 249
Odwiedzone miasta: 5
Znaleziony cykl: [1, 10, 9, 2, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 250
Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 9, 2, 8, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 251
Odwiedzone miasta: 5
Znaleziony cykl: [3, 10, 9, 8, 5]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 252
Odwiedzone miasta: 4
Znaleziony cykl: [3, 10, 9, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 253
Odwiedzone miasta: 7
Znaleziony cykl: [1, 7, 9, 4, 8, 6, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 254

Odwiedzone miasta: 3
Znaleziony cykl: [2, 8, 5]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 255
Odwiedzone miasta: 4
Znaleziony cykl: [2, 9, 4, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 256
Odwiedzone miasta: 6
Znaleziony cykl: [2, 6, 3, 10, 7, 4]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 257
Odwiedzone miasta: 4
Znaleziony cykl: [5, 8, 9, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 258
Odwiedzone miasta: 6
Znaleziony cykl: [2, 8, 5, 10, 9, 4]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 259
Odwiedzone miasta: 7
Znaleziony cykl: [2, 8, 5, 6, 7, 9, 4]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 260
Odwiedzone miasta: 4
Znaleziony cykl: [2, 10, 3, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 261
Odwiedzone miasta: 5

Znaleziony cykl: [2, 9, 10, 5, 8]
 Dodaję constraint dla cyklu długości 5
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 262
 Odwiedzone miasta: 6
 Znaleziony cykl: [2, 9, 7, 6, 5, 8]
 Dodaję constraint dla cyklu długości 6
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 263
 Odwiedzone miasta: 7
 Znaleziony cykl: [2, 8, 5, 3, 10, 7, 4]
 Dodaję constraint dla cyklu długości 7
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 264
 Odwiedzone miasta: 4
 Znaleziony cykl: [1, 10, 8, 6]
 Dodaję constraint dla cyklu długości 4
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 265
 Odwiedzone miasta: 7
 Znaleziony cykl: [1, 10, 9, 4, 2, 8, 6]
 Dodaję constraint dla cyklu długości 7
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 266
 Odwiedzone miasta: 3
 Znaleziony cykl: [3, 10, 4]
 Dodaję constraint dla cyklu długości 3
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 267
 Odwiedzone miasta: 7
 Znaleziony cykl: [1, 6, 8, 2, 7, 10, 3]
 Dodaję constraint dla cyklu długości 7
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 268
 Odwiedzone miasta: 5
 Znaleziony cykl: [3, 7, 9, 4, 6]

Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 269
Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 9, 2, 8, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 270
Odwiedzone miasta: 7
Znaleziony cykl: [1, 10, 7, 9, 4, 2, 8]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 271
Odwiedzone miasta: 4
Znaleziony cykl: [4, 8, 5, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 272
Odwiedzone miasta: 8
Znaleziony cykl: [1, 10, 7, 9, 4, 2, 8, 3]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 273
Odwiedzone miasta: 3
Znaleziony cykl: [2, 10, 9]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 274
Odwiedzone miasta: 6
Znaleziony cykl: [2, 8, 7, 10, 3, 5]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 275
Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 7, 9, 2, 8]
Dodaję constraint dla cyklu długości 6

Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 276
Odwiedzone miasta: 7
Znaleziony cykl: [1, 10, 7, 9, 2, 8, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 277
Odwiedzone miasta: 4
Znaleziony cykl: [3, 9, 8, 5]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 278
Odwiedzone miasta: 3
Znaleziony cykl: [3, 9, 6]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 279
Odwiedzone miasta: 6
Znaleziony cykl: [3, 7, 9, 8, 5, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 280
Odwiedzone miasta: 6
Znaleziony cykl: [3, 10, 7, 9, 8, 5]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 281
Odwiedzone miasta: 5
Znaleziony cykl: [3, 10, 7, 9, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 282
Odwiedzone miasta: 5
Znaleziony cykl: [2, 6, 10, 7, 4]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...

Zakończono iterację
Iteracja: 283
Odwiedzone miasta: 8
Znaleziony cykl: [2, 8, 5, 6, 3, 10, 9, 4]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 284
Odwiedzone miasta: 4
Znaleziony cykl: [4, 9, 10, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 285
Odwiedzone miasta: 6
Znaleziony cykl: [3, 7, 9, 4, 8, 5]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 286
Odwiedzone miasta: 7
Znaleziony cykl: [2, 8, 5, 10, 7, 9, 4]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 287
Odwiedzone miasta: 7
Znaleziony cykl: [2, 9, 10, 3, 6, 5, 8]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 288
Odwiedzone miasta: 5
Znaleziony cykl: [1, 9, 4, 2, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 289
Odwiedzone miasta: 6
Znaleziony cykl: [2, 9, 7, 10, 5, 8]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację

Iteracja: 290
Odwiedzone miasta: 6
Znaleziony cykl: [1, 7, 9, 8, 6, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 291
Odwiedzone miasta: 6
Znaleziony cykl: [1, 9, 4, 2, 8, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 292
Odwiedzone miasta: 4
Znaleziony cykl: [2, 5, 8, 4]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 293
Odwiedzone miasta: 8
Znaleziony cykl: [1, 10, 7, 9, 4, 2, 8, 6]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 294
Odwiedzone miasta: 4
Znaleziony cykl: [1, 9, 2, 8]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 295
Odwiedzone miasta: 5
Znaleziony cykl: [1, 9, 2, 8, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 296
Odwiedzone miasta: 7
Znaleziony cykl: [1, 10, 7, 9, 2, 8, 6]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 297

Odwiedzone miasta: 4
Znaleziony cykl: [2, 4, 10, 3]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 298
Odwiedzone miasta: 6
Znaleziony cykl: [2, 8, 5, 7, 9, 4]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 299
Odwiedzone miasta: 8
Znaleziony cykl: [1, 6, 8, 2, 4, 9, 10, 3]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 300
Odwiedzone miasta: 4
Znaleziony cykl: [2, 10, 7, 9]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 301
Odwiedzone miasta: 5
Znaleziony cykl: [2, 9, 7, 5, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 302
Odwiedzone miasta: 7
Znaleziony cykl: [1, 6, 8, 2, 9, 10, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 303
Odwiedzone miasta: 6
Znaleziony cykl: [1, 8, 2, 4, 9, 7]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 304
Odwiedzone miasta: 3

Znaleziony cykl: [3, 7, 6]
 Dodaję constraint dla cyklu długości 3
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 305
 Odwiedzone miasta: 7
 Znaleziony cykl: [1, 7, 9, 4, 2, 8, 3]
 Dodaję constraint dla cyklu długości 7
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 306
 Odwiedzone miasta: 7
 Znaleziony cykl: [2, 9, 4, 3, 6, 5, 8]
 Dodaję constraint dla cyklu długości 7
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 307
 Odwiedzone miasta: 3
 Znaleziony cykl: [1, 6, 3]
 Dodaję constraint dla cyklu długości 3
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 308
 Odwiedzone miasta: 5
 Znaleziony cykl: [1, 8, 2, 9, 7]
 Dodaję constraint dla cyklu długości 5
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 309
 Odwiedzone miasta: 6
 Znaleziony cykl: [1, 9, 4, 2, 8, 6]
 Dodaję constraint dla cyklu długości 6
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 310
 Odwiedzone miasta: 6
 Znaleziony cykl: [1, 7, 9, 2, 8, 3]
 Dodaję constraint dla cyklu długości 6
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 311
 Odwiedzone miasta: 9
 Znaleziony cykl: [2, 8, 5, 6, 3, 10, 7, 9, 4]

Dodaję constraint dla cyklu długości 9
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 312
Odwiedzone miasta: 5
Znaleziony cykl: [4, 9, 7, 10, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 313
Odwiedzone miasta: 5
Znaleziony cykl: [1, 9, 2, 8, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 314
Odwiedzone miasta: 6
Znaleziony cykl: [2, 9, 3, 6, 5, 8]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 315
Odwiedzone miasta: 4
Znaleziony cykl: [3, 10, 7, 4]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 316
Odwiedzone miasta: 8
Znaleziony cykl: [2, 9, 7, 10, 3, 6, 5, 8]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 317
Odwiedzone miasta: 4
Znaleziony cykl: [1, 10, 7, 8]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 318
Odwiedzone miasta: 7
Znaleziony cykl: [1, 6, 8, 2, 9, 4, 3]
Dodaję constraint dla cyklu długości 7

Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 319
Odwiedzone miasta: 5
Znaleziony cykl: [2, 7, 10, 3, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 320
Odwiedzone miasta: 4
Znaleziony cykl: [2, 8, 6, 5]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 321
Odwiedzone miasta: 5
Znaleziony cykl: [3, 7, 9, 8, 5]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 322
Odwiedzone miasta: 4
Znaleziony cykl: [3, 7, 9, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 323
Odwiedzone miasta: 6
Znaleziony cykl: [2, 6, 3, 10, 9, 4]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 324
Odwiedzone miasta: 7
Znaleziony cykl: [1, 7, 9, 4, 2, 8, 6]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 325
Odwiedzone miasta: 9
Znaleziony cykl: [1, 6, 8, 2, 4, 9, 7, 10, 3]
Dodaję constraint dla cyklu długości 9
Ponowne optymalizowanie...

Zakończono iterację
Iteracja: 326
Odwiedzone miasta: 3
Znaleziony cykl: [2, 9, 6]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 327
Odwiedzone miasta: 6
Znaleziony cykl: [1, 6, 8, 2, 9, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 328
Odwiedzone miasta: 6
Znaleziony cykl: [1, 7, 9, 2, 8, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 329
Odwiedzone miasta: 8
Znaleziony cykl: [1, 6, 8, 2, 9, 7, 10, 3]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 330
Odwiedzone miasta: 7
Znaleziony cykl: [2, 8, 5, 3, 10, 9, 4]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 331
Odwiedzone miasta: 5
Znaleziony cykl: [5, 8, 9, 10, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 332
Odwiedzone miasta: 6
Znaleziony cykl: [2, 9, 10, 3, 5, 8]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację

Iteracja: 333
Odwiedzone miasta: 4
Znaleziony cykl: [4, 9, 7, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 334
Odwiedzone miasta: 5
Znaleziony cykl: [1, 10, 7, 8, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 335
Odwiedzone miasta: 5
Znaleziony cykl: [1, 6, 7, 10, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 336
Odwiedzone miasta: 5
Znaleziony cykl: [2, 4, 7, 10, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 337
Odwiedzone miasta: 5
Znaleziony cykl: [2, 9, 4, 3, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 338
Odwiedzone miasta: 3
Znaleziony cykl: [2, 10, 3]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 339
Odwiedzone miasta: 5
Znaleziony cykl: [2, 6, 10, 9, 4]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 340

Odwiedzone miasta: 7
Znaleziony cykl: [2, 6, 3, 10, 7, 9, 4]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 341
Odwiedzone miasta: 6
Znaleziony cykl: [2, 9, 4, 3, 5, 8]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 342
Odwiedzone miasta: 8
Znaleziony cykl: [2, 8, 5, 6, 3, 7, 9, 4]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 343
Odwiedzone miasta: 8
Znaleziony cykl: [2, 8, 5, 3, 10, 7, 9, 4]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 344
Odwiedzone miasta: 6
Znaleziony cykl: [5, 8, 9, 7, 10, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 345
Odwiedzone miasta: 5
Znaleziony cykl: [2, 9, 10, 3, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 346
Odwiedzone miasta: 5
Znaleziony cykl: [2, 9, 3, 5, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 347
Odwiedzone miasta: 5

Znaleziony cykl: [2, 5, 6, 8, 4]
 Dodaję constraint dla cyklu długości 5
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 348
 Odwiedzone miasta: 7
 Znaleziony cykl: [2, 9, 7, 3, 6, 5, 8]
 Dodaję constraint dla cyklu długości 7
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 349
 Odwiedzone miasta: 7
 Znaleziony cykl: [2, 9, 7, 10, 3, 5, 8]
 Dodaję constraint dla cyklu długości 7
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 350
 Odwiedzone miasta: 4
 Znaleziony cykl: [1, 10, 9, 8]
 Dodaję constraint dla cyklu długości 4
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 351
 Odwiedzone miasta: 8
 Znaleziony cykl: [1, 7, 9, 4, 2, 8, 6, 3]
 Dodaję constraint dla cyklu długości 8
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 352
 Odwiedzone miasta: 7
 Znaleziony cykl: [1, 7, 9, 2, 8, 6, 3]
 Dodaję constraint dla cyklu długości 7
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 353
 Odwiedzone miasta: 6
 Znaleziony cykl: [2, 6, 10, 7, 9, 4]
 Dodaję constraint dla cyklu długości 6
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 354
 Odwiedzone miasta: 4
 Znaleziony cykl: [3, 10, 9, 4]

Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 355
Odwiedzone miasta: 4
Znaleziony cykl: [5, 10, 9, 8]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 356
Odwiedzone miasta: 5
Znaleziony cykl: [5, 8, 9, 7, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 357
Odwiedzone miasta: 3
Znaleziony cykl: [4, 9, 7]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 358
Odwiedzone miasta: 4
Znaleziony cykl: [2, 9, 3, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 359
Odwiedzone miasta: 6
Znaleziony cykl: [2, 9, 7, 10, 3, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 360
Odwiedzone miasta: 3
Znaleziony cykl: [1, 8, 3]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 361
Odwiedzone miasta: 5
Znaleziony cykl: [1, 10, 9, 8, 6]
Dodaję constraint dla cyklu długości 5

Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 362
Odwiedzone miasta: 4
Znaleziony cykl: [3, 8, 5, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 363
Odwiedzone miasta: 5
Znaleziony cykl: [1, 10, 7, 9, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 364
Odwiedzone miasta: 6
Znaleziony cykl: [2, 6, 3, 7, 9, 4]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 365
Odwiedzone miasta: 4
Znaleziony cykl: [2, 9, 10, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 366
Odwiedzone miasta: 5
Znaleziony cykl: [2, 6, 7, 9, 4]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 367
Odwiedzone miasta: 7
Znaleziony cykl: [2, 8, 5, 3, 7, 9, 4]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 368
Odwiedzone miasta: 6
Znaleziony cykl: [2, 9, 7, 3, 5, 8]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...

Zakończono iterację
Iteracja: 369
Odwiedzone miasta: 3
Znaleziony cykl: [3, 9, 4]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 370
Odwiedzone miasta: 4
Znaleziony cykl: [1, 6, 8, 3]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 371
Odwiedzone miasta: 5
Znaleziony cykl: [3, 10, 7, 9, 4]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 372
Odwiedzone miasta: 3
Znaleziony cykl: [3, 6, 5]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 373
Odwiedzone miasta: 4
Znaleziony cykl: [2, 7, 10, 3]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 374
Odwiedzone miasta: 5
Znaleziony cykl: [5, 10, 7, 9, 8]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 375
Odwiedzone miasta: 5
Znaleziony cykl: [2, 4, 9, 10, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację

Iteracja: 376
Odwiedzone miasta: 3
Znaleziony cykl: [2, 10, 7]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 377
Odwiedzone miasta: 3
Znaleziony cykl: [1, 9, 8]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 378
Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 7, 9, 8, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 379
Odwiedzone miasta: 3
Znaleziony cykl: [3, 10, 9]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 380
Odwiedzone miasta: 4
Znaleziony cykl: [5, 8, 9, 7]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 381
Odwiedzone miasta: 5
Znaleziony cykl: [2, 9, 7, 10, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 382
Odwiedzone miasta: 4
Znaleziony cykl: [1, 8, 9, 7]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 383

Odwiedzone miasta: 5
Znaleziony cykl: [2, 9, 7, 3, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 384
Odwiedzone miasta: 4
Znaleziony cykl: [1, 7, 10, 3]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 385
Odwiedzone miasta: 4
Znaleziony cykl: [1, 9, 8, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 386
Odwiedzone miasta: 4
Znaleziony cykl: [2, 9, 4, 3]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 387
Odwiedzone miasta: 5
Znaleziony cykl: [3, 10, 7, 5, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 388
Odwiedzone miasta: 6
Znaleziony cykl: [2, 4, 9, 7, 10, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 389
Odwiedzone miasta: 3
Znaleziony cykl: [3, 8, 5]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 390
Odwiedzone miasta: 4

Znaleziony cykl: [1, 4, 8, 3]
 Dodaję constraint dla cyklu długości 4
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 391
 Odwiedzone miasta: 4
 Znaleziony cykl: [2, 9, 10, 3]
 Dodaję constraint dla cyklu długości 4
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 392
 Odwiedzone miasta: 4
 Znaleziony cykl: [3, 10, 7, 9]
 Dodaję constraint dla cyklu długości 4
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 393
 Odwiedzone miasta: 5
 Znaleziony cykl: [1, 7, 9, 8, 6]
 Dodaję constraint dla cyklu długości 5
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 394
 Odwiedzone miasta: 5
 Znaleziony cykl: [4, 8, 5, 6, 7]
 Dodaję constraint dla cyklu długości 5
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 395
 Odwiedzone miasta: 3
 Znaleziony cykl: [1, 8, 4]
 Dodaję constraint dla cyklu długości 3
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 396
 Odwiedzone miasta: 4
 Znaleziony cykl: [2, 9, 7, 6]
 Dodaję constraint dla cyklu długości 4
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 397
 Odwiedzone miasta: 5
 Znaleziony cykl: [3, 6, 5, 8, 4]

Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 398
Odwiedzone miasta: 4
Znaleziony cykl: [3, 7, 9, 4]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 399
Odwiedzone miasta: 5
Znaleziony cykl: [1, 6, 8, 4, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 400
Odwiedzone miasta: 4
Znaleziony cykl: [1, 7, 8, 3]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 401
Odwiedzone miasta: 4
Znaleziony cykl: [1, 6, 8, 4]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 402
Odwiedzone miasta: 3
Znaleziony cykl: [2, 9, 3]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 403
Odwiedzone miasta: 5
Znaleziony cykl: [2, 9, 7, 10, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 404
Odwiedzone miasta: 5
Znaleziony cykl: [1, 5, 8, 10, 3]
Dodaję constraint dla cyklu długości 5

Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 405
Odwiedzone miasta: 3
Znaleziony cykl: [2, 10, 6]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 406
Odwiedzone miasta: 5
Znaleziony cykl: [2, 4, 9, 7, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 407
Odwiedzone miasta: 5
Znaleziony cykl: [1, 7, 4, 8, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 408
Odwiedzone miasta: 4
Znaleziony cykl: [4, 8, 5, 7]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 409
Odwiedzone miasta: 5
Znaleziony cykl: [1, 10, 4, 8, 5]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 410
Odwiedzone miasta: 3
Znaleziony cykl: [3, 9, 7]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 411
Odwiedzone miasta: 4
Znaleziony cykl: [1, 8, 4, 7]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...

Zakończono iterację
Iteracja: 412
Odwiedzone miasta: 4
Znaleziony cykl: [3, 5, 8, 4]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 413
Odwiedzone miasta: 3
Znaleziony cykl: [2, 7, 4]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 414
Odwiedzone miasta: 3
Znaleziony cykl: [3, 6, 4]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 415
Odwiedzone miasta: 6
Znaleziony cykl: [1, 6, 5, 8, 10, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 416
Odwiedzone miasta: 3
Znaleziony cykl: [4, 10, 9]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 417
Odwiedzone miasta: 5
Znaleziony cykl: [1, 7, 4, 8, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 418
Odwiedzone miasta: 6
Znaleziony cykl: [1, 5, 8, 4, 10, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację

Iteracja: 419
Odwiedzone miasta: 4
Znaleziony cykl: [2, 9, 7, 3]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 420
Odwiedzone miasta: 5
Znaleziony cykl: [3, 7, 8, 5, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 421
Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 4, 8, 5, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 422
Odwiedzone miasta: 5
Znaleziony cykl: [1, 7, 8, 6, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 423
Odwiedzone miasta: 5
Znaleziony cykl: [2, 8, 3, 6, 5]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 424
Odwiedzone miasta: 3
Znaleziony cykl: [3, 10, 5]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 425
Odwiedzone miasta: 6
Znaleziony cykl: [3, 7, 4, 8, 5, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 426

Odwiedzone miasta: 4
Znaleziony cykl: [2, 7, 10, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 427
Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 7, 4, 8, 5]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 428
Odwiedzone miasta: 4
Znaleziony cykl: [4, 10, 7, 9]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 429
Odwiedzone miasta: 3
Znaleziony cykl: [2, 6, 4]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 430
Odwiedzone miasta: 6
Znaleziony cykl: [1, 5, 8, 7, 10, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 431
Odwiedzone miasta: 7
Znaleziony cykl: [1, 6, 5, 8, 4, 10, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 432
Odwiedzone miasta: 6
Znaleziony cykl: [1, 7, 4, 8, 6, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 433
Odwiedzone miasta: 6

Znaleziony cykl: [2, 8, 5, 6, 7, 4]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 434
Odwiedzone miasta: 4
Znaleziony cykl: [1, 8, 2, 4]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 435
Odwiedzone miasta: 5
Znaleziony cykl: [1, 4, 2, 8, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 436
Odwiedzone miasta: 7
Znaleziony cykl: [1, 5, 8, 4, 7, 10, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 437
Odwiedzone miasta: 4
Znaleziony cykl: [3, 7, 8, 5]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 438
Odwiedzone miasta: 3
Znaleziony cykl: [1, 8, 2]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 439
Odwiedzone miasta: 5
Znaleziony cykl: [2, 8, 7, 6, 5]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 440
Odwiedzone miasta: 4
Znaleziony cykl: [1, 8, 2, 3]

Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 441
Odwiedzone miasta: 4
Znaleziony cykl: [5, 8, 10, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 442
Odwiedzone miasta: 6
Znaleziony cykl: [2, 8, 5, 6, 3, 4]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 443
Odwiedzone miasta: 7
Znaleziony cykl: [1, 10, 7, 4, 8, 5, 6]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 444
Odwiedzone miasta: 5
Znaleziony cykl: [1, 6, 4, 10, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 445
Odwiedzone miasta: 5
Znaleziony cykl: [3, 10, 4, 5, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 446
Odwiedzone miasta: 5
Znaleziony cykl: [1, 6, 8, 2, 4]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 447
Odwiedzone miasta: 4
Znaleziony cykl: [1, 8, 2, 6]
Dodaję constraint dla cyklu długości 4

Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 448
Odwiedzone miasta: 5
Znaleziony cykl: [1, 6, 2, 8, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 449
Odwiedzone miasta: 7
Znaleziony cykl: [1, 6, 5, 8, 7, 10, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 450
Odwiedzone miasta: 4
Znaleziony cykl: [2, 8, 3, 5]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 451
Odwiedzone miasta: 5
Znaleziony cykl: [3, 7, 4, 8, 5]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 452
Odwiedzone miasta: 5
Znaleziony cykl: [1, 10, 8, 2, 5]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 453
Odwiedzone miasta: 6
Znaleziony cykl: [1, 6, 8, 2, 4, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 454
Odwiedzone miasta: 4
Znaleziony cykl: [3, 7, 4, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...

Zakończono iterację
Iteracja: 455
Odwiedzone miasta: 5
Znaleziony cykl: [2, 8, 5, 7, 4]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 456
Odwiedzone miasta: 8
Znaleziony cykl: [1, 6, 5, 8, 4, 7, 10, 3]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 457
Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 4, 2, 8, 5]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 458
Odwiedzone miasta: 5
Znaleziony cykl: [1, 8, 2, 4, 7]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 459
Odwiedzone miasta: 4
Znaleziony cykl: [2, 8, 7, 5]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 460
Odwiedzone miasta: 6
Znaleziony cykl: [1, 7, 4, 2, 8, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 461
Odwiedzone miasta: 3
Znaleziony cykl: [4, 10, 7]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację

Iteracja: 462
Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 9, 4, 8, 5]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 463
Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 8, 2, 5, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 464
Odwiedzone miasta: 5
Znaleziony cykl: [2, 8, 5, 3, 4]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 465
Odwiedzone miasta: 6
Znaleziony cykl: [1, 5, 2, 8, 10, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 466
Odwiedzone miasta: 3
Znaleziony cykl: [5, 10, 8]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 467
Odwiedzone miasta: 3
Znaleziony cykl: [6, 8, 7]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 468
Odwiedzone miasta: 6
Znaleziony cykl: [1, 7, 4, 2, 8, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 469

Odwiedzone miasta: 6
Znaleziony cykl: [1, 6, 4, 7, 10, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 470
Odwiedzone miasta: 6
Znaleziony cykl: [3, 10, 7, 4, 5, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 471
Odwiedzone miasta: 7
Znaleziony cykl: [1, 10, 4, 2, 8, 5, 6]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 472
Odwiedzone miasta: 7
Znaleziony cykl: [1, 5, 8, 4, 9, 10, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 473
Odwiedzone miasta: 5
Znaleziony cykl: [1, 5, 6, 10, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 474
Odwiedzone miasta: 7
Znaleziony cykl: [1, 10, 7, 9, 4, 8, 5]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 475
Odwiedzone miasta: 7
Znaleziony cykl: [1, 5, 8, 2, 4, 10, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 476
Odwiedzone miasta: 4

Znaleziony cykl: [2, 6, 3, 4]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 477
Odwiedzone miasta: 4
Znaleziony cykl: [1, 8, 2, 7]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 478
Odwiedzone miasta: 5
Znaleziony cykl: [1, 7, 2, 8, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 479
Odwiedzone miasta: 7
Znaleziony cykl: [1, 10, 9, 4, 8, 5, 6]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 480
Odwiedzone miasta: 5
Znaleziony cykl: [5, 8, 7, 10, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 481
Odwiedzone miasta: 7
Znaleziony cykl: [1, 10, 7, 4, 2, 8, 5]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 482
Odwiedzone miasta: 7
Znaleziony cykl: [1, 6, 5, 2, 8, 10, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 483
Odwiedzone miasta: 5
Znaleziony cykl: [1, 9, 4, 8, 5]

Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 484
Odwiedzone miasta: 7
Znaleziony cykl: [2, 8, 5, 6, 3, 7, 4]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 485
Odwiedzone miasta: 5
Znaleziony cykl: [1, 7, 2, 8, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 486
Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 7, 8, 2, 5]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 487
Odwiedzone miasta: 6
Znaleziony cykl: [1, 5, 8, 4, 9, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 488
Odwiedzone miasta: 8
Znaleziony cykl: [1, 5, 8, 4, 9, 7, 10, 3]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 489
Odwiedzone miasta: 3
Znaleziony cykl: [4, 7, 6]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 490
Odwiedzone miasta: 6
Znaleziony cykl: [2, 5, 6, 3, 10, 4]
Dodaję constraint dla cyklu długości 6

Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 491
Odwiedzone miasta: 6
Znaleziony cykl: [2, 8, 7, 3, 6, 5]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 492
Odwiedzone miasta: 8
Znaleziony cykl: [1, 6, 5, 8, 4, 9, 10, 3]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 493
Odwiedzone miasta: 8
Znaleziony cykl: [1, 10, 7, 9, 4, 8, 5, 6]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 494
Odwiedzone miasta: 8
Znaleziony cykl: [1, 6, 5, 8, 2, 4, 10, 3]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 495
Odwiedzone miasta: 7
Znaleziony cykl: [1, 7, 4, 2, 8, 6, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 496
Odwiedzone miasta: 6
Znaleziony cykl: [1, 7, 9, 4, 8, 5]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 497
Odwiedzone miasta: 4
Znaleziony cykl: [4, 10, 6, 5]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...

Zakończono iterację
Iteracja: 498
Odwiedzone miasta: 3
Znaleziony cykl: [2, 6, 3]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 499
Odwiedzone miasta: 6
Znaleziony cykl: [1, 5, 8, 9, 10, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 500
Odwiedzone miasta: 3
Znaleziony cykl: [1, 8, 6]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 501
Odwiedzone miasta: 8
Znaleziony cykl: [1, 10, 7, 4, 2, 8, 5, 6]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 502
Odwiedzone miasta: 8
Znaleziony cykl: [1, 5, 8, 2, 4, 7, 10, 3]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 503
Odwiedzone miasta: 6
Znaleziony cykl: [1, 9, 4, 8, 5, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 504
Odwiedzone miasta: 5
Znaleziony cykl: [2, 5, 6, 10, 4]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację

Iteracja: 505
 Odwiedzone miasta: 3
 Znaleziony cykl: [1, 10, 6]
 Dodaję constraint dla cyklu długości 3
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 506
 Odwiedzone miasta: 7
 Znaleziony cykl: [1, 10, 7, 8, 2, 5, 6]
 Dodaję constraint dla cyklu długości 7
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 507
 Odwiedzone miasta: 7
 Znaleziony cykl: [1, 6, 5, 8, 4, 9, 3]
 Dodaję constraint dla cyklu długości 7
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 508
 Odwiedzone miasta: 9
 Znaleziony cykl: [1, 6, 5, 8, 4, 9, 7, 10, 3]
 Dodaję constraint dla cyklu długości 9
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 509
 Odwiedzone miasta: 6
 Znaleziony cykl: [1, 6, 4, 9, 10, 3]
 Dodaję constraint dla cyklu długości 6
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 510
 Odwiedzone miasta: 7
 Znaleziony cykl: [1, 5, 2, 8, 7, 10, 3]
 Dodaję constraint dla cyklu długości 7
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 511
 Odwiedzone miasta: 6
 Znaleziony cykl: [3, 10, 9, 4, 5, 6]
 Dodaję constraint dla cyklu długości 6
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 512

Odwiedzone miasta: 4
Znaleziony cykl: [5, 10, 7, 8]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 513
Odwiedzone miasta: 5
Znaleziony cykl: [1, 5, 8, 9, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 514
Odwiedzone miasta: 7
Znaleziony cykl: [1, 5, 8, 9, 7, 10, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 515
Odwiedzone miasta: 4
Znaleziony cykl: [1, 10, 4, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 516
Odwiedzone miasta: 6
Znaleziony cykl: [2, 8, 5, 3, 7, 4]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 517
Odwiedzone miasta: 7
Znaleziony cykl: [1, 7, 9, 4, 8, 5, 6]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 518
Odwiedzone miasta: 6
Znaleziony cykl: [1, 7, 2, 8, 6, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 519
Odwiedzone miasta: 7

Znaleziony cykl: [1, 6, 5, 8, 9, 10, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 520
Odwiedzone miasta: 7
Znaleziony cykl: [1, 5, 8, 4, 9, 7, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 521
Odwiedzone miasta: 6
Znaleziony cykl: [1, 6, 2, 4, 10, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 522
Odwiedzone miasta: 5
Znaleziony cykl: [2, 9, 4, 6, 5]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 523
Odwiedzone miasta: 5
Znaleziony cykl: [2, 8, 7, 3, 5]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 524
Odwiedzone miasta: 7
Znaleziony cykl: [2, 5, 6, 3, 10, 7, 4]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 525
Odwiedzone miasta: 4
Znaleziony cykl: [4, 9, 6, 5]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 526
Odwiedzone miasta: 5
Znaleziony cykl: [2, 10, 3, 6, 5]

Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 527
Odwiedzone miasta: 4
Znaleziony cykl: [1, 10, 8, 5]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 528
Odwiedzone miasta: 7
Znaleziony cykl: [1, 10, 9, 4, 2, 8, 5]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 529
Odwiedzone miasta: 9
Znaleziony cykl: [1, 6, 5, 8, 2, 4, 7, 10, 3]
Dodaję constraint dla cyklu długości 9
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 530
Odwiedzone miasta: 5
Znaleziony cykl: [1, 6, 4, 9, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 531
Odwiedzone miasta: 5
Znaleziony cykl: [2, 6, 3, 7, 4]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 532
Odwiedzone miasta: 5
Znaleziony cykl: [4, 7, 10, 6, 5]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 533
Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 9, 2, 8, 5]
Dodaję constraint dla cyklu długości 6

Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 534
Odwiedzone miasta: 7
Znaleziony cykl: [1, 6, 4, 9, 7, 10, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 535
Odwiedzone miasta: 5
Znaleziony cykl: [3, 9, 4, 5, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 536
Odwiedzone miasta: 7
Znaleziony cykl: [3, 10, 7, 9, 4, 5, 6]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 537
Odwiedzone miasta: 4
Znaleziony cykl: [2, 6, 7, 4]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 538
Odwiedzone miasta: 8
Znaleziony cykl: [1, 6, 5, 2, 8, 7, 10, 3]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 539
Odwiedzone miasta: 6
Znaleziony cykl: [1, 6, 5, 8, 9, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 540
Odwiedzone miasta: 8
Znaleziony cykl: [1, 6, 5, 8, 9, 7, 10, 3]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...

Zakończono iterację
Iteracja: 541
Odwiedzone miasta: 6
Znaleziony cykl: [2, 5, 6, 10, 7, 4]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 542
Odwiedzone miasta: 8
Znaleziony cykl: [1, 7, 9, 4, 8, 5, 6, 3]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 543
Odwiedzone miasta: 8
Znaleziony cykl: [1, 10, 7, 9, 4, 2, 8, 5]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 544
Odwiedzone miasta: 3
Znaleziony cykl: [1, 8, 7]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 545
Odwiedzone miasta: 6
Znaleziony cykl: [1, 5, 8, 9, 7, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 546
Odwiedzone miasta: 7
Znaleziony cykl: [1, 10, 7, 9, 2, 8, 5]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 547
Odwiedzone miasta: 5
Znaleziony cykl: [1, 10, 8, 5, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację

Iteracja: 548
Odwiedzone miasta: 8
Znaleziony cykl: [1, 10, 9, 4, 2, 8, 5, 6]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 549
Odwiedzone miasta: 5
Znaleziony cykl: [1, 10, 7, 4, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 550
Odwiedzone miasta: 5
Znaleziony cykl: [1, 6, 9, 10, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 551
Odwiedzone miasta: 8
Znaleziony cykl: [1, 5, 8, 2, 4, 9, 10, 3]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 552
Odwiedzone miasta: 7
Znaleziony cykl: [1, 10, 9, 2, 8, 5, 6]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 553
Odwiedzone miasta: 4
Znaleziony cykl: [3, 10, 7, 5]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 554
Odwiedzone miasta: 7
Znaleziony cykl: [1, 5, 8, 2, 9, 10, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 555

Odwiedzone miasta: 7
Znaleziony cykl: [1, 6, 2, 4, 7, 10, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 556
Odwiedzone miasta: 5
Znaleziony cykl: [1, 6, 2, 10, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 557
Odwiedzone miasta: 6
Znaleziony cykl: [1, 9, 4, 2, 8, 5]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 558
Odwiedzone miasta: 5
Znaleziony cykl: [3, 10, 9, 5, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 559
Odwiedzone miasta: 4
Znaleziony cykl: [1, 7, 8, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 560
Odwiedzone miasta: 6
Znaleziony cykl: [1, 7, 9, 4, 6, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 561
Odwiedzone miasta: 5
Znaleziony cykl: [1, 9, 2, 8, 5]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 562
Odwiedzone miasta: 6

Znaleziony cykl: [3, 7, 9, 4, 5, 6]
 Dodaję constraint dla cyklu długości 6
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 563
 Odwiedzone miasta: 5
 Znaleziony cykl: [1, 10, 4, 2, 6]
 Dodaję constraint dla cyklu długości 5
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 564
 Odwiedzone miasta: 7
 Znaleziony cykl: [1, 5, 8, 2, 9, 4, 3]
 Dodaję constraint dla cyklu długości 7
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 565
 Odwiedzone miasta: 4
 Znaleziony cykl: [1, 10, 4, 3]
 Dodaję constraint dla cyklu długości 4
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 566
 Odwiedzone miasta: 7
 Znaleziony cykl: [1, 7, 9, 8, 5, 6, 3]
 Dodaję constraint dla cyklu długości 7
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 567
 Odwiedzone miasta: 9
 Znaleziony cykl: [1, 10, 7, 9, 4, 2, 8, 5, 6]
 Dodaję constraint dla cyklu długości 9
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 568
 Odwiedzone miasta: 4
 Znaleziony cykl: [1, 6, 9, 3]
 Dodaję constraint dla cyklu długości 4
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 569
 Odwiedzone miasta: 6
 Znaleziony cykl: [1, 6, 9, 7, 10, 3]

Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 570
Odwiedzone miasta: 6
Znaleziony cykl: [2, 7, 10, 3, 6, 5]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 571
Odwiedzone miasta: 7
Znaleziony cykl: [1, 7, 9, 4, 2, 8, 5]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 572
Odwiedzone miasta: 9
Znaleziony cykl: [1, 5, 8, 2, 4, 9, 7, 10, 3]
Dodaję constraint dla cyklu długości 9
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 573
Odwiedzone miasta: 7
Znaleziony cykl: [2, 5, 6, 3, 10, 9, 4]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 574
Odwiedzone miasta: 8
Znaleziony cykl: [1, 10, 7, 9, 2, 8, 5, 6]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 575
Odwiedzone miasta: 6
Znaleziony cykl: [1, 5, 8, 2, 9, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 576
Odwiedzone miasta: 6
Znaleziony cykl: [1, 7, 9, 2, 8, 5]
Dodaję constraint dla cyklu długości 6

Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 577
Odwiedzone miasta: 4
Znaleziony cykl: [2, 9, 6, 5]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 578
Odwiedzone miasta: 8
Znaleziony cykl: [1, 5, 8, 2, 9, 7, 10, 3]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 579
Odwiedzone miasta: 9
Znaleziony cykl: [1, 6, 5, 8, 2, 4, 9, 10, 3]
Dodaję constraint dla cyklu długości 9
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 580
Odwiedzone miasta: 4
Znaleziony cykl: [2, 7, 3, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 581
Odwiedzone miasta: 5
Znaleziony cykl: [4, 9, 10, 6, 5]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 582
Odwiedzone miasta: 8
Znaleziony cykl: [1, 6, 5, 8, 2, 9, 10, 3]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 583
Odwiedzone miasta: 5
Znaleziony cykl: [1, 10, 7, 8, 5]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...

Zakończono iterację
Iteracja: 584
Odwiedzone miasta: 4
Znaleziony cykl: [3, 9, 5, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 585
Odwiedzone miasta: 6
Znaleziony cykl: [3, 10, 7, 9, 5, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 586
Odwiedzone miasta: 7
Znaleziony cykl: [1, 9, 4, 2, 8, 5, 6]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 587
Odwiedzone miasta: 3
Znaleziony cykl: [2, 4, 3]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 588
Odwiedzone miasta: 6
Znaleziony cykl: [1, 9, 2, 8, 5, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 589
Odwiedzone miasta: 4
Znaleziony cykl: [2, 9, 4, 5]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 590
Odwiedzone miasta: 6
Znaleziony cykl: [2, 9, 4, 3, 6, 5]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację

Iteracja: 591
Odwiedzone miasta: 4
Znaleziony cykl: [1, 7, 6, 3]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 592
Odwiedzone miasta: 6
Znaleziony cykl: [2, 5, 6, 10, 9, 4]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 593
Odwiedzone miasta: 5
Znaleziony cykl: [1, 10, 4, 2, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 594
Odwiedzone miasta: 8
Znaleziony cykl: [2, 5, 6, 3, 10, 7, 9, 4]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 595
Odwiedzone miasta: 8
Znaleziony cykl: [1, 6, 5, 8, 2, 9, 4, 3]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 596
Odwiedzone miasta: 8
Znaleziony cykl: [1, 7, 9, 4, 2, 8, 5, 6]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 597
Odwiedzone miasta: 10
Znaleziony cykl: [1, 6, 5, 8, 2, 4, 9, 7, 10, 3]
Dodaję constraint dla cyklu długości 10
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 598

Odwiedzone miasta: 6
Znaleziony cykl: [4, 9, 7, 10, 6, 5]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 599
Odwiedzone miasta: 7
Znaleziony cykl: [1, 6, 5, 8, 2, 9, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 600
Odwiedzone miasta: 7
Znaleziony cykl: [1, 7, 9, 2, 8, 5, 6]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 601
Odwiedzone miasta: 6
Znaleziony cykl: [2, 9, 10, 3, 6, 5]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 602
Odwiedzone miasta: 9
Znaleziony cykl: [1, 6, 5, 8, 2, 9, 7, 10, 3]
Dodaję constraint dla cyklu długości 9
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 603
Odwiedzone miasta: 5
Znaleziony cykl: [1, 10, 9, 4, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 604
Odwiedzone miasta: 5
Znaleziony cykl: [1, 7, 9, 6, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 605
Odwiedzone miasta: 6

Znaleziony cykl: [1, 10, 7, 8, 5, 6]
 Dodaję constraint dla cyklu długości 6
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 606
 Odwiedzone miasta: 6
 Znaleziony cykl: [1, 6, 2, 7, 10, 3]
 Dodaję constraint dla cyklu długości 6
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 607
 Odwiedzone miasta: 6
 Znaleziony cykl: [1, 10, 7, 4, 2, 6]
 Dodaję constraint dla cyklu długości 6
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 608
 Odwiedzone miasta: 7
 Znaleziony cykl: [1, 6, 2, 4, 9, 10, 3]
 Dodaję constraint dla cyklu długości 7
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 609
 Odwiedzone miasta: 5
 Znaleziony cykl: [1, 10, 7, 4, 3]
 Dodaję constraint dla cyklu długości 5
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 610
 Odwiedzone miasta: 8
 Znaleziony cykl: [1, 5, 8, 2, 4, 9, 7, 3]
 Dodaję constraint dla cyklu długości 8
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 611
 Odwiedzone miasta: 7
 Znaleziony cykl: [1, 5, 8, 2, 9, 7, 3]
 Dodaję constraint dla cyklu długości 7
 Ponowne optymalizowanie...
 Zakończono iterację
 Iteracja: 612
 Odwiedzone miasta: 4
 Znaleziony cykl: [1, 5, 6, 3]

Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 613
Odwiedzone miasta: 7
Znaleziony cykl: [2, 5, 6, 10, 7, 9, 4]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 614
Odwiedzone miasta: 5
Znaleziony cykl: [3, 7, 9, 5, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 615
Odwiedzone miasta: 5
Znaleziony cykl: [4, 9, 7, 6, 5]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 616
Odwiedzone miasta: 5
Znaleziony cykl: [1, 10, 9, 8, 5]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 617
Odwiedzone miasta: 5
Znaleziony cykl: [2, 9, 3, 6, 5]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 618
Odwiedzone miasta: 7
Znaleziony cykl: [2, 9, 7, 10, 3, 6, 5]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 619
Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 7, 9, 4, 6]
Dodaję constraint dla cyklu długości 6

Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 620
Odwiedzone miasta: 6
Znaleziony cykl: [1, 6, 2, 9, 4, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 621
Odwiedzone miasta: 8
Znaleziony cykl: [1, 6, 2, 4, 9, 7, 10, 3]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 622
Odwiedzone miasta: 3
Znaleziony cykl: [3, 7, 4]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 623
Odwiedzone miasta: 7
Znaleziony cykl: [2, 5, 6, 3, 7, 9, 4]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 624
Odwiedzone miasta: 3
Znaleziony cykl: [5, 8, 6]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 625
Odwiedzone miasta: 5
Znaleziony cykl: [2, 5, 3, 10, 4]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 626
Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 7, 4, 2, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...

Zakończono iterację
Iteracja: 627
Odwiedzone miasta: 5
Znaleziony cykl: [2, 9, 10, 6, 5]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 628
Odwiedzone miasta: 9
Znaleziony cykl: [1, 7, 9, 4, 2, 8, 5, 6, 3]
Dodaję constraint dla cyklu długości 9
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 629
Odwiedzone miasta: 6
Znaleziony cykl: [2, 5, 6, 7, 9, 4]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 630
Odwiedzone miasta: 4
Znaleziony cykl: [1, 10, 2, 3]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 631
Odwiedzone miasta: 6
Znaleziony cykl: [1, 6, 2, 9, 10, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 632
Odwiedzone miasta: 8
Znaleziony cykl: [1, 7, 9, 2, 8, 5, 6, 3]
Dodaję constraint dla cyklu długości 8
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 633
Odwiedzone miasta: 4
Znaleziony cykl: [1, 5, 8, 3]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację

Iteracja: 634
Odwiedzone miasta: 4
Znaleziony cykl: [1, 9, 4, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 635
Odwiedzone miasta: 6
Znaleziony cykl: [1, 5, 6, 7, 10, 3]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 636
Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 7, 9, 8, 5]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 637
Odwiedzone miasta: 6
Znaleziony cykl: [1, 10, 9, 8, 5, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 638
Odwiedzone miasta: 5
Znaleziony cykl: [1, 7, 9, 4, 6]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 639
Odwiedzone miasta: 4
Znaleziony cykl: [2, 4, 7, 3]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 640
Odwiedzone miasta: 6
Znaleziony cykl: [2, 9, 7, 10, 6, 5]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 641

Odwiedzone miasta: 4
Znaleziony cykl: [2, 5, 10, 4]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 642
Odwiedzone miasta: 3
Znaleziony cykl: [2, 9, 5]
Dodaję constraint dla cyklu długości 3
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 643
Odwiedzone miasta: 5
Znaleziony cykl: [1, 6, 2, 9, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 644
Odwiedzone miasta: 4
Znaleziony cykl: [1, 9, 8, 5]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 645
Odwiedzone miasta: 7
Znaleziony cykl: [1, 6, 2, 9, 7, 10, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 646
Odwiedzone miasta: 6
Znaleziony cykl: [2, 9, 7, 3, 6, 5]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 647
Odwiedzone miasta: 4
Znaleziony cykl: [6, 10, 7, 9]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 648
Odwiedzone miasta: 6

Znaleziony cykl: [1, 10, 9, 4, 2, 6]
Dodaję constraint dla cyklu długości 6
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 649
Odwiedzone miasta: 5
Znaleziony cykl: [1, 10, 9, 4, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 650
Odwiedzone miasta: 5
Znaleziony cykl: [1, 6, 5, 8, 3]
Dodaję constraint dla cyklu długości 5
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 651
Odwiedzone miasta: 7
Znaleziony cykl: [1, 7, 9, 4, 2, 6, 3]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 652
Odwiedzone miasta: 4
Znaleziony cykl: [3, 7, 5, 6]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 653
Odwiedzone miasta: 4
Znaleziony cykl: [3, 10, 4, 5]
Dodaję constraint dla cyklu długości 4
Ponowne optymalizowanie...
Zakończono iterację
Iteracja: 654
Odwiedzone miasta: 7
Znaleziony cykl: [1, 10, 7, 9, 8, 5, 6]
Dodaję constraint dla cyklu długości 7
Ponowne optymalizowanie...
Zakończono iterację

```

function decode_optional_route(route, d, cost_item_city)
    n = length(route)
    k = size(cost_item_city, 1)

    assigned = zeros(Int, k)

    for p in 1:k
        best = argmin(cost_item_city[p, route])
        assigned[p] = route[best]
    end

    visited = Set(assigned)
    sub_route = [c for c in route if c in visited]

    travel = 0.0
    for i in 1:length(sub_route)-1
        travel += d[sub_route[i], sub_route[i+1]]
    end

    if length(sub_route) > 1
        travel += d[sub_route[end], sub_route[1]]
    end

    shopping = sum(minimum(cost_item_city[p, :]) for p in 1:k)

    return travel + shopping, visited
end

```

```

start = randperm(n5)

cost_fn = route -> decode_optional_route(route, d5, c5)[1]

time_ga = @elapsed result = Evolutionary.optimize(
    cost_fn,
    start,
    GA(populationSize=1000),
    Evolutionary.Options(iterations=300)
)

best_route = result.minimizer
ga_cost, visited = decode_optional_route(best_route, d5, c5)

```

```
println("GA koszt: ", ga_cost)
println("Odwiedzone miasta: ", visited)
println("Pominięte miasta: ", setdiff(1:n5, visited))
```

```
println("==== PORÓWNANIE ====")
println("MIP koszt: ", objective_value(opt_model5))
println("GA koszt: ", ga_cost)
```

```
using Plots
```

```
function plot_solution_pretty(model, X, Y, n)
    x_val = value.(model[:x])
    y_val = value.(model[:y])

    p = plot(title="TPP 5.0 - rozwiązanie MIP", legend=false)

    # Krawędzie
    for i in 1:n
        for j in i+1:n
            if x_val[i,j] > 0.5
                plot!([X[i], X[j]], [Y[i], Y[j]], lw=2)
            end
        end
    end

    # Pominięte miasta (szare)
    skipped = [i for i in 1:n if y_val[i] < 0.5]
    scatter!(X[skipped], Y[skipped], markersize=6)

    # Odwiedzone miasta (większe)
    visited = [i for i in 1:n if y_val[i] > 0.5]
    scatter!(X[visited], Y[visited], markersize=10)

    # Numery
    for i in 1:n
        annotate!(X[i], Y[i] + 2, text(string(i), 8))
    end

    return p
end

plot_solution_pretty(opt_model5, X5, Y5, n5)
```

```

function plot_ga_vs_mip(route, visited_ga, model, X, Y, n)

    p1 = plot_solution_pretty(model, X, Y, n)

    # GA
    sub = [c for c in route if c in visited_ga]

    p2 = plot(title="GA - trasa", legend=false)

    if length(sub) > 1
        closed = vcat(sub, sub[1])
        plot!(X[closed], Y[closed], lw=2)
    end

    scatter!(X, Y, markersize=6)

    for i in 1:n
        annotate!(X[i], Y[i] + 2, text(string(i), 8))
    end

    plot(p1, p2, layout=(1,2), size=(900,400))
end

plot_ga_vs_mip(best_route, visited, opt_model5, X5, Y5, n5)

```